INDEX

S.No	Date	Name of the Experiment	Page No	Remarks
1		**Lists, Links and Images**		
	28.07.2026	a. Write a HTML program to explain the working of lists.	3	
	28.07.2026	b. Write a HTML program for hyperlinks using <a>, href, target.	6	
		c. Create a HTML document with your image and a friend's image; each image links to their profile.	8	
		d. HTML program to use thumbnails (100x100 px), each a link to the full image. Gallery.	10	
2		**HTML Tables, Forms and Frames**		
		a. HTML program for tables (tags: <table>, <tr>, <td>, border, rowspan, colspan).	12	
		b. Table timetable using <caption>, cell spacing, padding, border, rowspan, colspan.	15	
		c. Registration form: text, password, number, DOB, checkboxes, radio, list box, <select>, <option>, two buttons, table layout.	18	
		d. Frames dividing page in 3: image, paragraph, hyperlink. Use "no frame" attribute.	22	
3		**HTML 5 and Cascading Style Sheets, Types of CSS**		
		a. HTML using <footer>, <header>, <main>, <nav>, <section>, , .	25	
		b. Embed audio and video in HTML webpage.	27	
		c. Apply inline, internal, external styles—identify selector, property, value.	29	
4		**Selector Forms**		
		a. Apply different selector forms: i. Simple selector (element, id, class, group, universal) ii. Combinator selector (descendant, child, adjacent sibling, general sibling) iii. Pseudo-class selector iv. Pseudo-element selector v. Attribute selector	32	
5		**CSS: Color, Background, Font, Text, Box Model**		
		a. Reference color in CSS different ways.	42	
		b. Background image halfway down page, tilting horizontally, remains on scroll.	44	
		c. Use: i. font-size ii. font-weight iii. font-style iv. text-decoration v. text-transform vi. text-align	46	

S.No	Date	Name of the Experiment	Page No	Remarks
		d. CSS Box Model: i. Content ii. Border iii. Margin iv. Padding	49	
6		**Applying JavaScript: Internal/External, I/O, Type Conversion**		
		a. Embed internal and external JavaScript.	51	
		b. JavaScript output methods.	53	
		c. JavaScript input methods.	55	
		d. Prompt for name/age, display in table, note voter eligibility.	57	
7		**JavaScript Pre-defined and User-defined Objects**		
		a. Document object: properties and methods.	59	
		b. Window object: properties and methods.	61	
		c. Array object: properties and methods.	63	
		d. Math object: properties and methods.	65	
		e. String object: properties and methods.	67	
		f. Regex object: properties and methods.	69	
		g. Date object: properties and methods.	71	
		h. User-defined object: properties, methods, accessors, constructors, display.	73	
8	18.09.2026	**JavaScript Conditional Statements and Loops**		
		a. Input three integers, show largest/"equal numbers" in HTML dialog.	75	
		b. Display days of week (switch case).	77	
		c. Print 1–10 using for, while, do-while loops.	79	
		d. Print object data using for-in, for-each, for-of loops.	81	
		e. Armstrong number check (e.g., 153).	83	
		f. Show denomination breakdown for a deposit (100's, 50's, etc.).	85	
9	25.10.2026	**JavaScript Functions and Events**		
		a. Function for: i. Factorial ii. Fibonacci series iii. Prime numbers iv. Palindrome check	87	
		b. HTML with textbox and 4 buttons: when clicked, call and show i. Factorial ii. Fibonacci series iii. Prime numbers iv. Palindrome check	90	
		c. Validate registration page: i. Name (start with letter, alphanumeric, min 6 chars) ii. Mobile (10 digits) iii. Email (xxxx@xxxx.xxx)	93	

Experiment: 1a **Date:28.07.2026**

Working of Lists

Write a HTML program, to explain the working of lists.

Aim:

To write a HTML program, to explain the working of lists (ordered list, unordered list, nested lists, an unordered list inside an ordered list and an ordered list inside an unordered list).

Description:

ordered list: An ordered list in HTML is a list of items that are arranged in a specific sequence and are automatically numbered (1, 2, 3…) by the browser. It is created using the <ol> tag, and each item is written inside an <li> tag.

unordered list: An unordered list is a list of items that are not in any sequence. It is created using the <ul> tag, and the items are marked with bullets (●, ○, etc.) instead of numbers.

nested list: A nested list in HTML is a list that contains another list (ordered or unordered) inside one of its <li> items. It is used to show sub-items or a hierarchy within a main list. This program shows an unordered list inside an ordered list and an ordered list inside an unordered list.

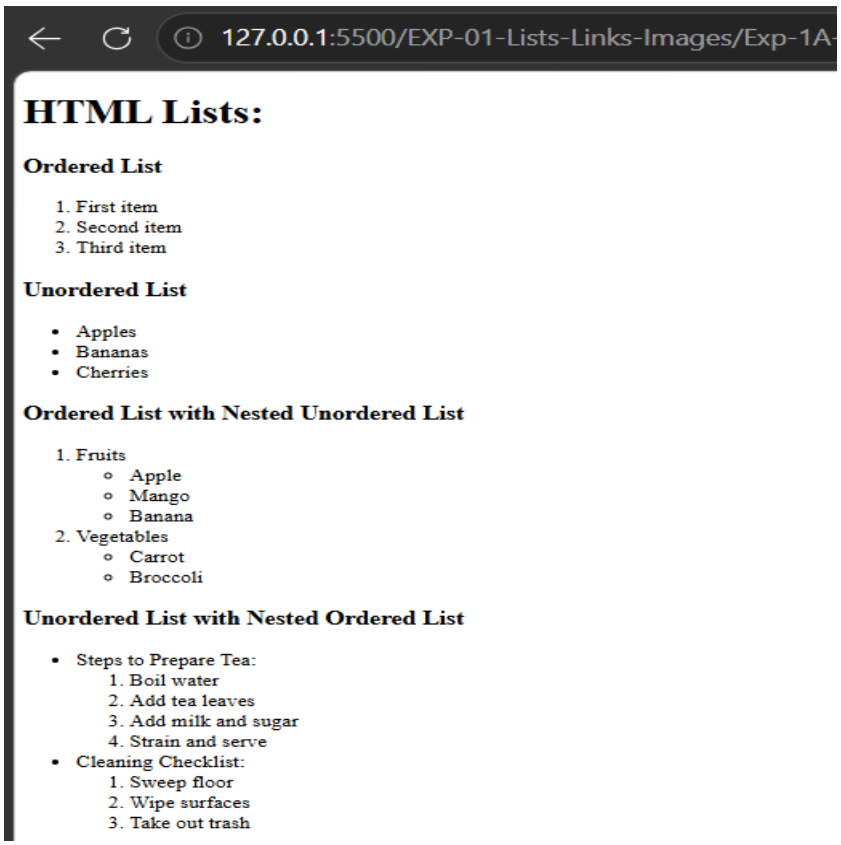
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>HTML Lists</title>
  </head>
  <body>
    <h1>HTML Lists:</h1>
    <h3>Ordered List</h3>
    <ol>
      <li>First item</li>
```

```
  <li>Second item</li>
  <li>Third item</li>
</ol>
<h3>Unordered List</h3>
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Cherries</li>
</ul>
<h3>Ordered List with Nested Unordered List</h3>
<ol>
  <li>
    Fruits
    <ul>
      <li>Apple</li>
      <li>Mango</li>
      <li>Banana</li>
    </ul>
  </li>
  <li>
    Vegetables
    <ul>
      <li>Carrot</li>
      <li>Broccoli</li>
    </ul>
  </li>
</ol>
<h3>Unordered List with Nested Ordered List</h3>
<ul>
  <li>
    Steps to Prepare Tea:
    <ol>
      <li>Boil water</li>
      <li>Add tea leaves</li>
```

```
        <li>Add milk and sugar</li>
        <li>Strain and serve</li>
      </ol>
    </li>
    <li>
      Cleaning Checklist:
      <ol>
        <li>Sweep floor</li>
        <li>Wipe surfaces</li>
        <li>Take out trash</li>
      </ol>
    </li>
  </ul>
 </body>
</html>
```

Output:



Experiment: 1b **Date:**

Working of Hyperlinks

Write a HTML program, to explain the working of hyperlinks using <a> tag and href, target Attributes.

Aim:

To write a HTML program, to explain the working of hyperlinks using <a> tag and href, target Attributes.

Description:

<a> tag: The <a> tag is called the anchor tag. It is used to create hyperlinks in HTML, allowing users to navigate from one page to another, open an external website, or jump to a section within the same page.

href attribute: The href (Hypertext reference) attribute inside <a> specifies the destination URL or location where the link will go. It can be an external website address or a path to another page of the same site.

target attribute: The target attribute in the <a> tag specifies where to open the linked document when the user clicks on the hyperlink. Common values are _blank (opens in a new tab) and _self (opens in the same tab, the default).

Program:

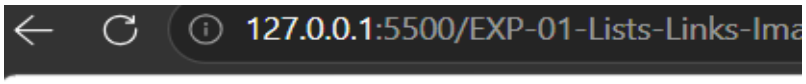
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>HTML Hyperlinks</title>
  </head>
  <body>
    <h1>Hyperlinks</h1>
    <h3>Open in a new tab</h3>
    <a href="https://www.google.com" target="_blank">Go to Google</a>

    <h3>Open in the same tab</h3>
```

```
    <a href="https://www.youtube.com" target="_self">YouTube</a>

    <h3>Link to another page</h3>
    <a href="../Exp-1A-Lists/Exp-1A-Lists.html">EXP-1-Lists</a>
  </body>
</html>
```

Output:



Hyperlinks

Open in a new tab

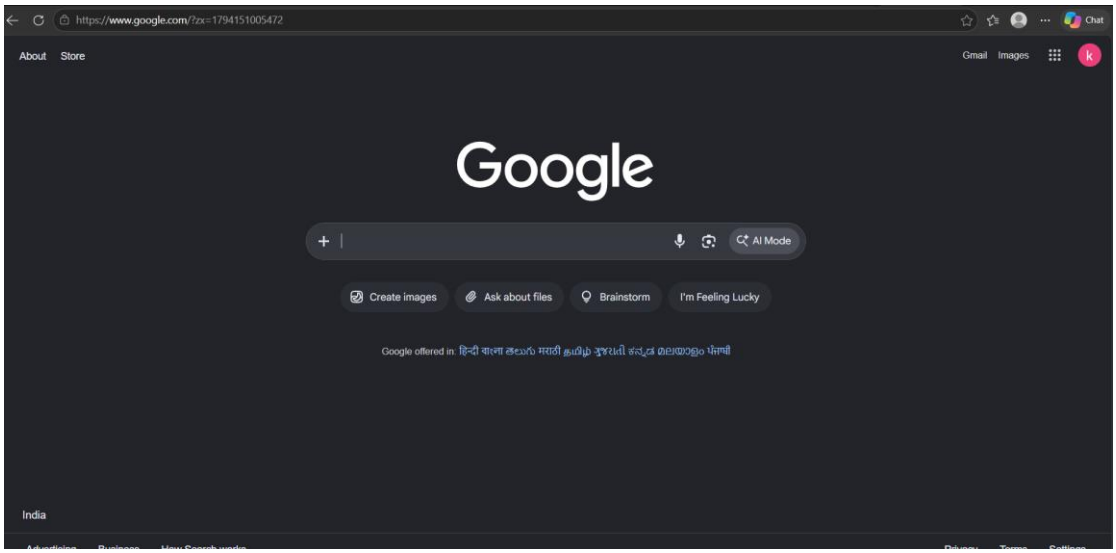
Go to Google

Open in the same tab

YouTube

Link to another page

EXP-1-Lists



Experiment: 1c **Date:**

Profile Images with Links

Create a HTML document that has your image and your friend's image with a specific height and width. Also when clicked on the images it should navigate to their respective profiles.

Aim:

To Create a HTML document that has your image and your friend's image with a specific height and width. Also when clicked on the images it should navigate to their respective profiles.

Description:

<img> tag: Displays an image. The src attribute gives the image file, alt gives alternate text, and the width and height attributes set the size of the image.

Image as a link: When the <img> tag is placed inside an <a> tag, the image becomes clickable. The href attribute of <a> holds the profile link, and target="_blank" opens that profile in a new tab.

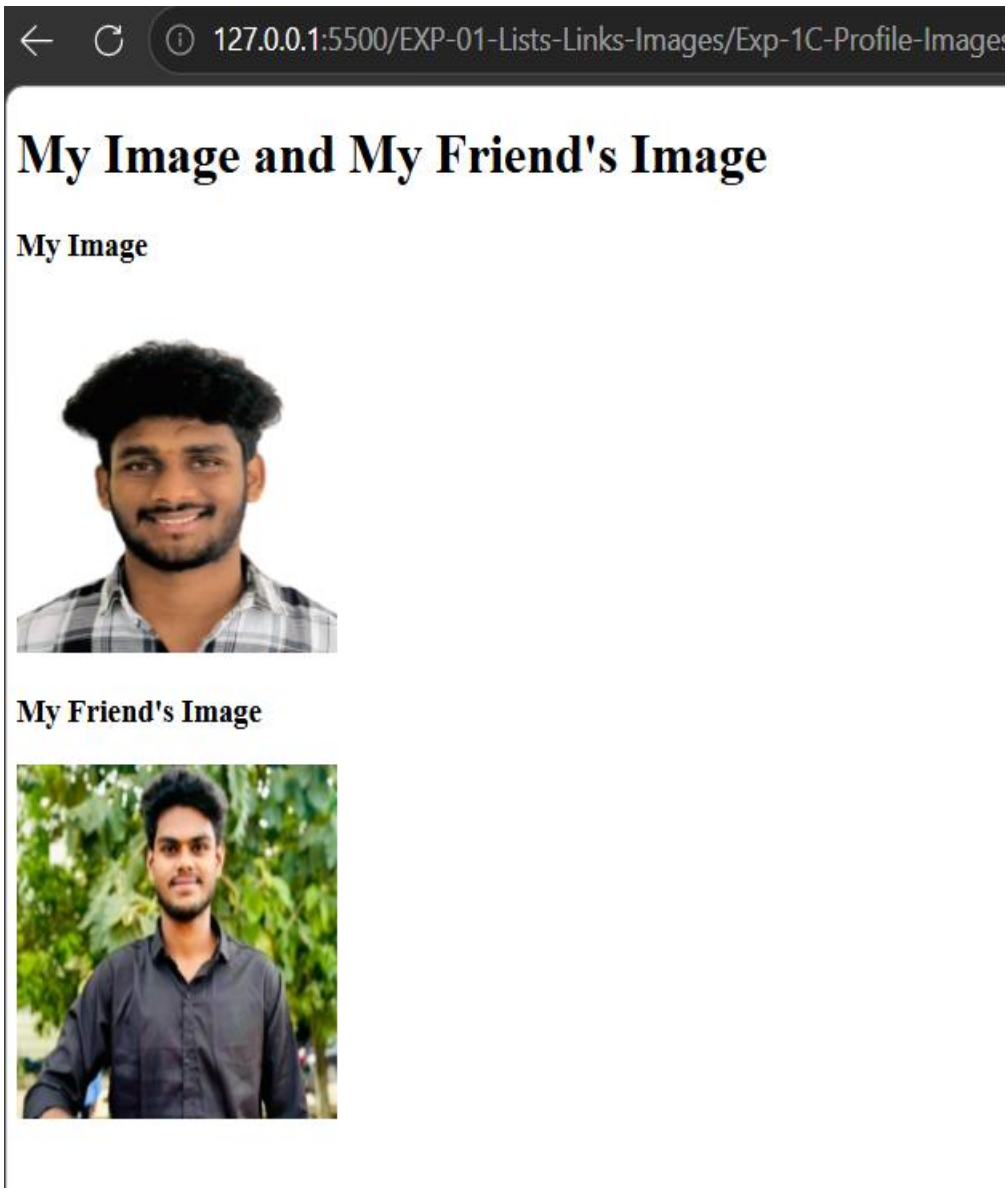
Program:

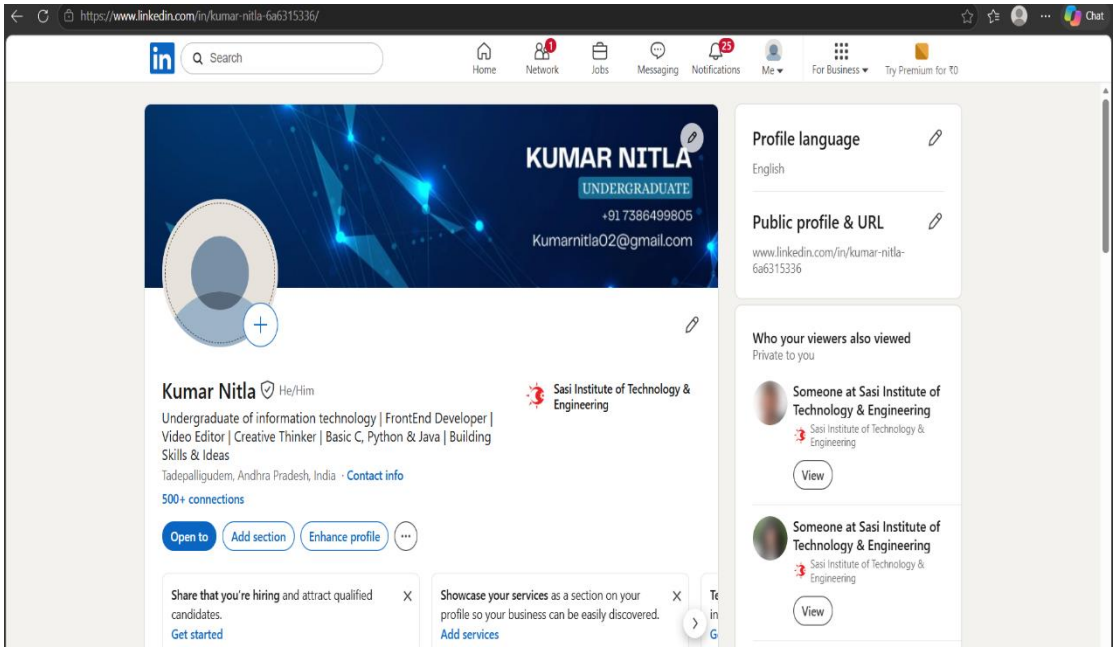
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Profile Images</title>
  </head>
  <body>
    <h1>My Image and My Friend's Image</h1>

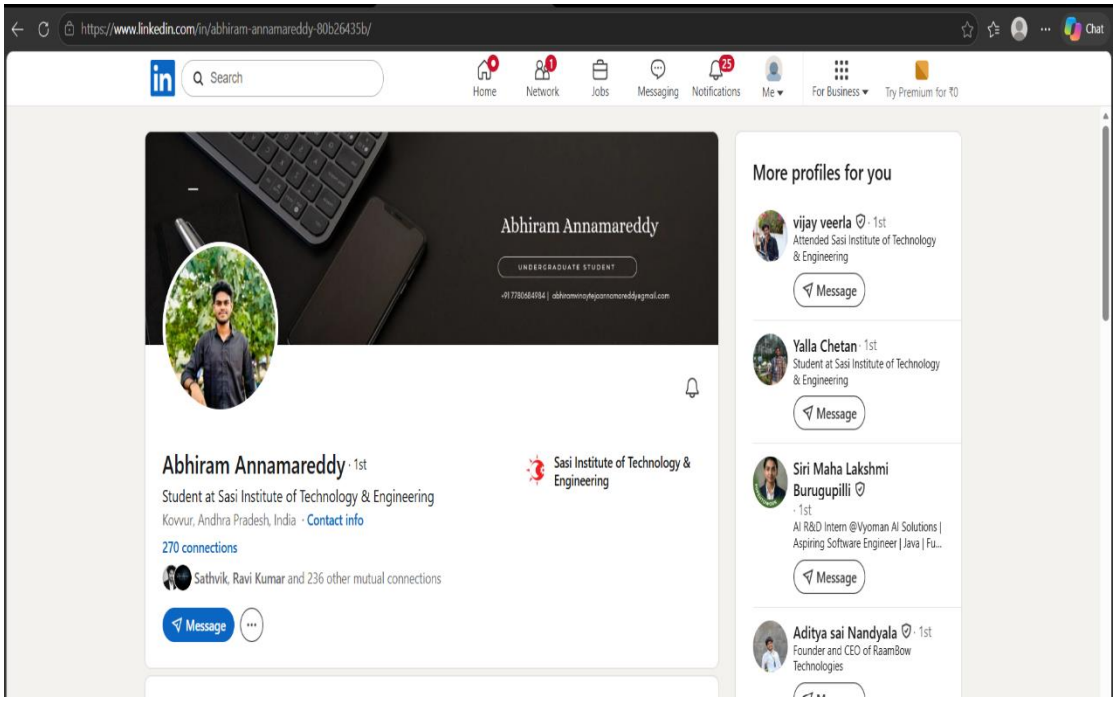
    <h3>My Image</h3>
    <a href="https://www.linkedin.com/feed/" target="_blank">
      <img src="me.jpg" alt="My Image" width="200" height="200" />
    </a>
<h3>My Friend's Image</h3>
```

```
    <a
      href="https://www.linkedin.com/in/abhiram-annamareddy-80b26435b/"
      target="_blank"
    >
      <img src="friend.jpg" alt="Friend Image" width="200" height="200" />
    </a>
  </body>
</html>
```

Output:




https://www.linkedin.com/in/kumar-nitla-6a6315336/
Search
Home
Network
Jobs
Messaging
Notifications
Me
For Business
Try Premium for ₹0
KUMAR NITLA
UNDERGRADUATE
+91 7386499805
Kumarnitla02@gmail.com
Kumar Nitla He/Him
Undergraduate of information technology | FrontEnd Developer | Video Editor | Creative Thinker | Basic C, Python & Java | Building Skills & Ideas
Tadepalligudem, Andhra Pradesh, India · Contact info
500+ connections
Open to
Add section
Enhance profile
Sasi Institute of Technology & Engineering
Share that you're hiring and attract qualified candidates.
Get started
Showcase your services as a section on your profile so your business can be easily discovered.
Add services
Profile language
English
Public profile & URL
www.linkedin.com/in/kumar-nitla-6a6315336
Who your viewers also viewed
Private to you
Someone at Sasi Institute of Technology & Engineering
Sasi Institute of Technology & Engineering
View
Someone at Sasi Institute of Technology & Engineering
Sasi Institute of Technology & Engineering
View


https://www.linkedin.com/in/abhiram-annamareddy-80b26435b/
Search
Home
Network
Jobs
Messaging
Notifications
Me
For Business
Try Premium for ₹0
Abhiram Annamareddy
UNDERGRADUATE STUDENT
Abhiram Annamareddy · 1st
Student at Sasi Institute of Technology & Engineering
Kovvur, Andhra Pradesh, India · Contact info
270 connections
Sathvik, Ravi Kumar and 236 other mutual connections
Message
Sasi Institute of Technology & Engineering
More profiles for you
vijay veerla · 1st
Attended Sasi Institute of Technology & Engineering
Message
Yalla Chetan · 1st
Student at Sasi Institute of Technology & Engineering
Message
Siri Maha Lakshmi Burugupilli · 1st
AI R&D Intern @Vyoman AI Solutions | Aspiring Software Engineer | Java | Fu...
Message
Aditya sai Nandyala · 1st
Founder and CEO of Raam8ow Technologies

Experiment: 1d **Date:**

Thumbnail Image Gallery

Write a HTML program, in such a way that, rather than placing large images on a page, the preferred technique is to use thumbnails by setting the height and width parameters to something like 100*100 pixels. Each thumbnail image is also a link to a full sized version of the image. Create an image gallery using this technique.

Aim:

To write a HTML program, in such a way that, rather than placing large images on a page, the preferred technique is to use thumbnails by setting the height and width parameters to something like 100*100 pixels. Each thumbnail image is also a link to a full sized version of the image. Create an image gallery using this technique.

Description:

Thumbnail: A thumbnail is a smaller version of an image (here 100x100 pixels) obtained by setting the width and height attributes of the <img> tag.

Linked thumbnail: Each thumbnail is placed inside an <a> tag whose href points to the full-sized image, so clicking the thumbnail opens the original image.

Advantage: Small images keep the page compact and easy to scan, while the user can still view the original image by clicking the thumbnail.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Thumbnail Gallery</title>
  </head>
  <body>
    <h1>Image Gallery</h1>
    <p>Click on a thumbnail to see the full image</p>
    <a href="pic1.jpg"
```

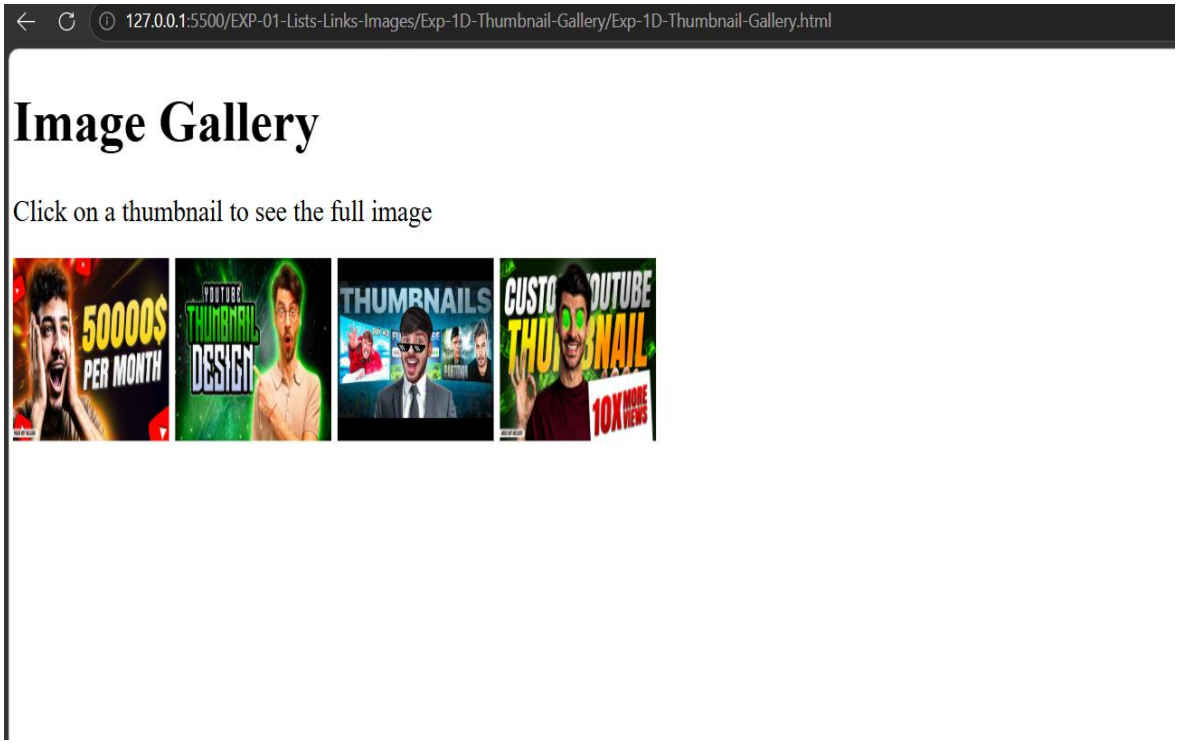
```
    ><img src="pic1.jpg" alt="Picture 1" width="100" height="100"
   /></a>

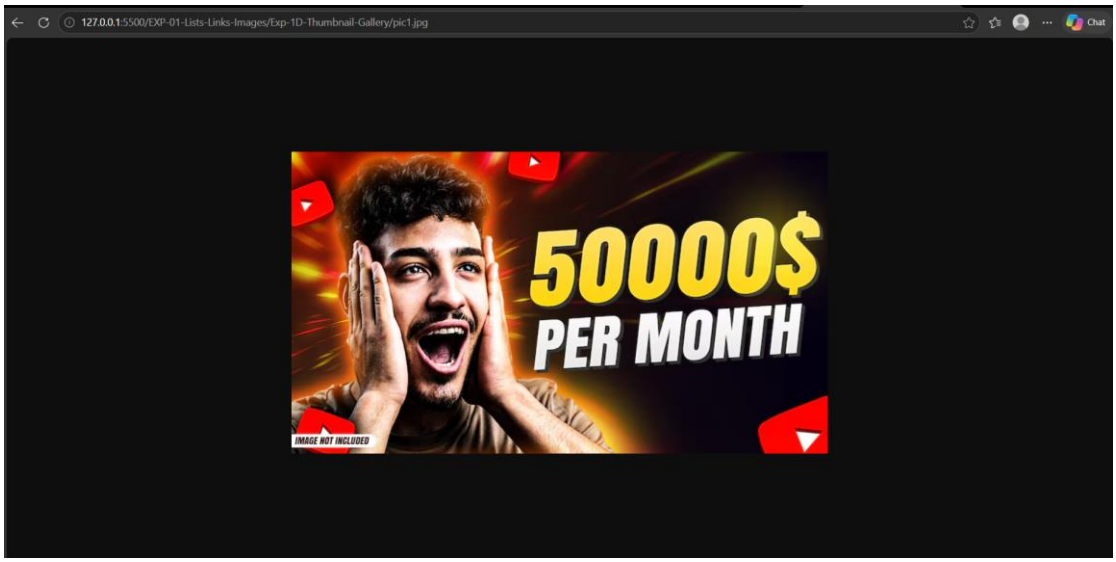
   <a href="pic2.jpg"
    ><img src="pic2.jpg" alt="Picture 2" width="100" height="100"
   /></a>

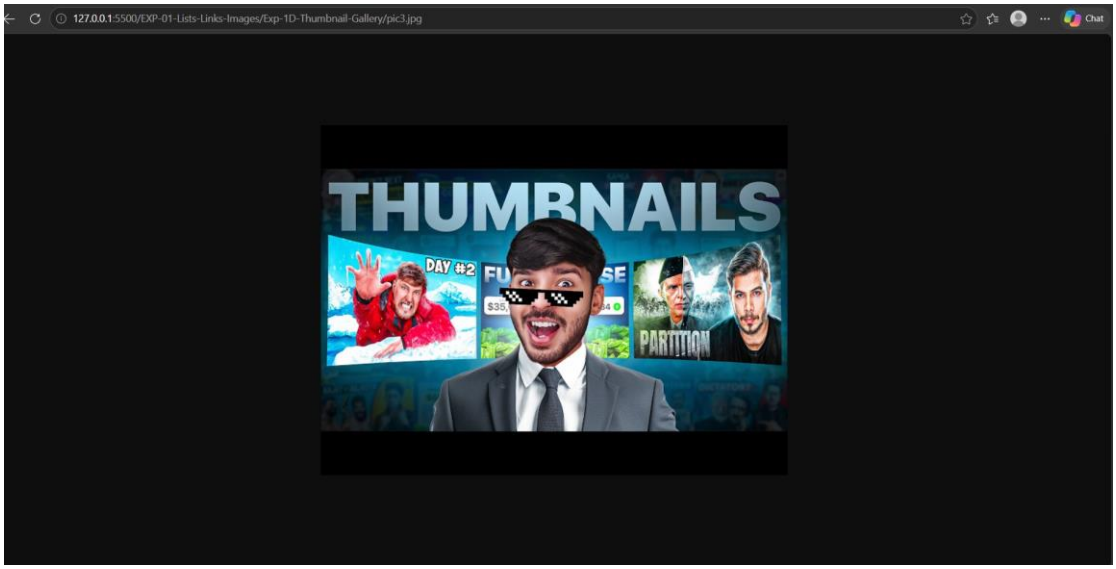
   <a href="pic3.jpg"
    ><img src="pic3.jpg" alt="Picture 3" width="100" height="100"
   /></a>

   <a href="pic4.jpg"
    ><img src="pic4.jpg" alt="Picture 4" width="100" height="100"
   /></a>
  </body>
</html>
```

Output:




127.0.0.1:5500/EXP-01-Lists-Links-Images/Exp-1D-Thumbnail-Gallery/pic1.jpg
Chat
50000$
PER MONTH
IMAGE NOT INCLUDED


127.0.0.1:5500/EXP-01-Lists-Links-Images/Exp-1D-Thumbnail-Gallery/pic3.jpg
Chat
THUMBNAILS
DAY #2
PARTITION

Experiment: 2a **Date:**

HTML Tables

Write a HTML program for tables using the tags <table>, <tr>, <td> and the attributes border, rowspan and colspan.

Aim:

To write a HTML program to create tables using the <table>, <tr>, <th>, <td> tags and the border, rowspan and colspan attributes.

Description:

<table>: Defines a table. The border attribute draws a border around the table and its cells.

<tr>, <th>, <td>: <tr> defines a table row, <th> defines a bold, centered header cell and <td> defines a normal data cell.

rowspan: Makes a cell extend downwards across the given number of rows.

colspan: Makes a cell extend sideways across the given number of columns.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>HTML Tables</title>
  </head>
  <body>
    <h1>HTML Tables Demonstration</h1>
    <h3>Simple Table</h3>
    <table border="1">
      <tr>
        <th>Roll No</th>
        <th>Name</th>
        <th>Marks</th>
      </tr>
```

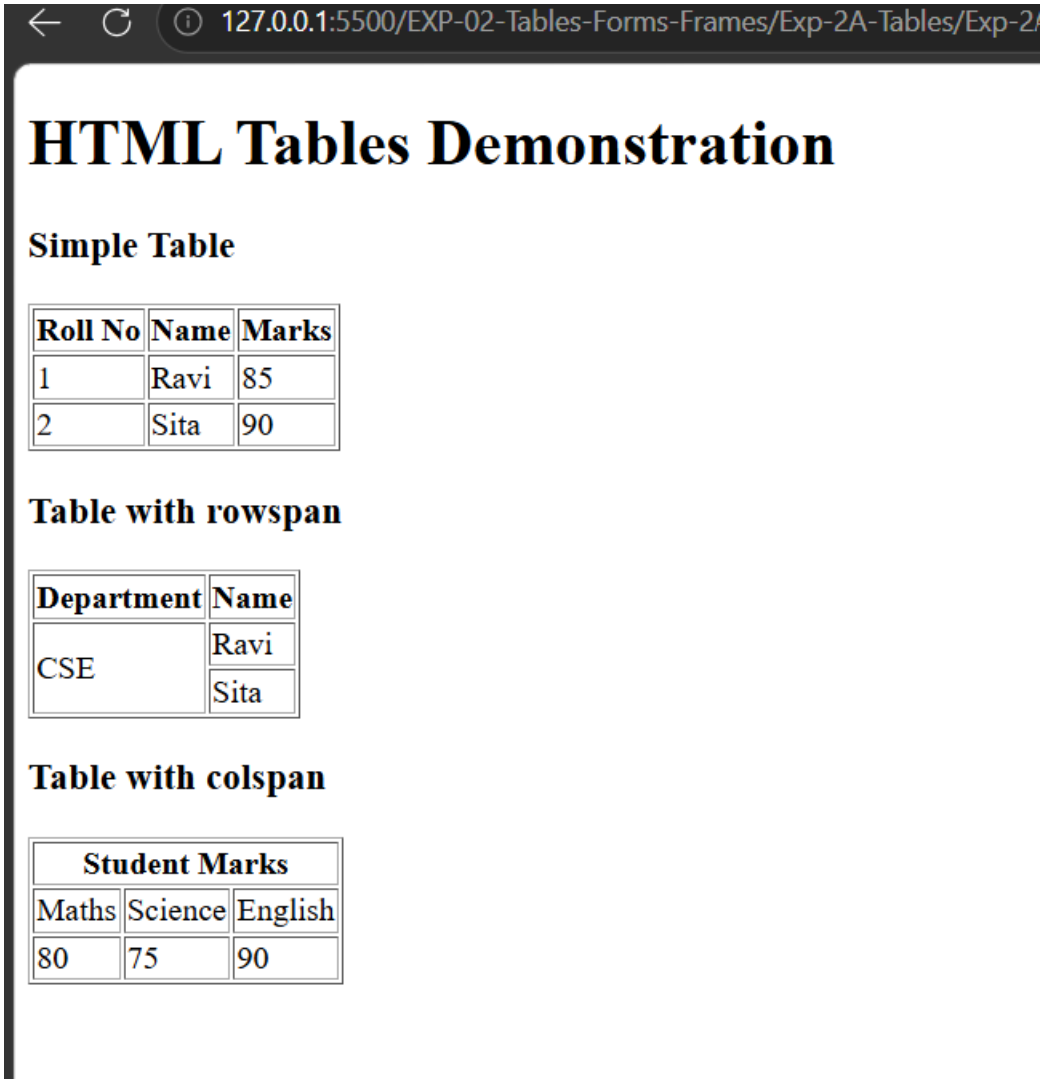
```
  <tr>
    <td>1</td>
    <td>Ravi</td>
    <td>85</td>
  </tr>
  <tr>
    <td>2</td>
    <td>Sita</td>
    <td>90</td>
  </tr>
</table>

<h3>Table with rowspan</h3>
<table border="1">
  <tr>
    <th>Department</th>
    <th>Name</th>
  </tr>
  <tr>
    <td rowspan="2">CSE</td>
    <td>Ravi</td>
  </tr>
  <tr>
    <td>Sita</td>
  </tr>
</table>

<h3>Table with colspan</h3>
<table border="1">
  <tr>
    <th colspan="3">Student Marks</th>
  </tr>
  <tr>
    <td>Maths</td>
```

```
      <td>Science</td>
      <td>English</td>
    </tr>
    <tr>
      <td>80</td>
      <td>75</td>
      <td>90</td>
    </tr>
  </table>
 </body>
</html>
```

Output:



Experiment: 2b **Date:**

Class Timetable

Create a table to show your class timetable using <caption>, cellspacing, cellpadding, border, rowspan and colspan.

Aim:

To create a class timetable using a HTML table with the <caption> tag and the cellspacing, cellpadding, border, rowspan and colspan attributes.

Description:

<caption>: Gives a title to the table; it is displayed above the table.

cellspacing: Sets the space between adjacent cells.

cellpadding: Sets the space between the cell border and the cell content.

border: Sets the thickness of the table border.

rowspan / colspan: rowspan merges cells vertically (used for the Lunch Break cell) and colspan merges cells horizontally (used for the Lab Session cell).

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Class Timetable</title>
  </head>
  <body>
    <table border="2" cellspacing="4" cellpadding="10" align="center">
      <caption>
        <b>Class Timetable</b>
      </caption>
      <tr>
        <th>Day</th>
        <th>9:00 - 10:00</th>
        <th>10:00 - 11:00</th>
```

```
  <th>11:00 - 12:00</th>
  <th>12:00 - 1:00</th>
  <th>2:00 - 4:00</th>
</tr>
<tr>
  <td>Monday</td>
  <td>Maths</td>
  <td>Physics</td>
  <td>Chemistry</td>
  <td rowspan="5">Lunch Break</td>
  <td>English</td>
</tr>
<tr>
  <td>Tuesday</td>
  <td>Physics</td>
  <td>Maths</td>
  <td>English</td>
  <td>Chemistry</td>
</tr>
<tr>
  <td>Wednesday</td>
  <td>Chemistry</td>
  <td>English</td>
  <td>Maths</td>
  <td>Physics</td>
</tr>
<tr>
  <td>Thursday</td>
  <td colspan="3">Lab Session</td>
  <td>Maths</td>
</tr>
<tr>
  <td>Friday</td>
  <td>English</td>
```

```
        <td>Chemistry</td>
        <td>Physics</td>
        <td>Sports</td>
      </tr>
    </table>
  </body>
</html>
```

Output:



Class Timetable

Year: B.Tech.		Semester: II		Branch: IT	Section: B	Room No.: 14104		w.e.f: 30-01-2025
Period / Day	1	2	3	Break	4	5	Break	6
	09:00-10:00	10:00-11:00	11:00-12:00	12:00-01:00	01:00-02:00	02:00-03:00	03:00-03:10	03:10-04:30
Monday	CE	DS	BCME	Lunch	← DS LAB →			
Tuesday	DS	← CE/CHE LAB →			BCME	CE		DE&VC
Wednesday	BCME	CHE	DE&VC		← VERBAL TRAINING →			
Thursday	DE&VC	CHE	DE&VC		BCME	DS		SPORTS
Friday	← EWS →				CE	CHE		BCME
Saturday	CHE	DE&VC	DS		DS	← CE/CHE LAB →		

Experiment: 2c **Date:**

Registration Form

Create a registration form with text, password, number and date of birth fields, checkboxes, radio buttons, a list box, <select> and <option> tags, and two buttons (Submit and Reset). Use a table to lay out the form.

Aim:

To create a registration form using HTML form controls (text box, password, number, date, radio buttons, checkboxes, list box, drop-down list, Submit and Reset buttons) laid out using a table.

Description:

<form>: Container that holds the input controls used to collect data from the user.

<input> types: text (single-line text), password (hidden characters), number (numeric value), date (date picker for date of birth), radio (select one option), checkbox (select many options), submit and reset (buttons).

<select> and <option>: <select> creates a drop-down list and each <option> is one item in it. With the size and multiple attributes it becomes a list box in which several items can be selected.

Table layout: The labels and controls are placed in the rows and columns of a <table> to keep the form aligned.

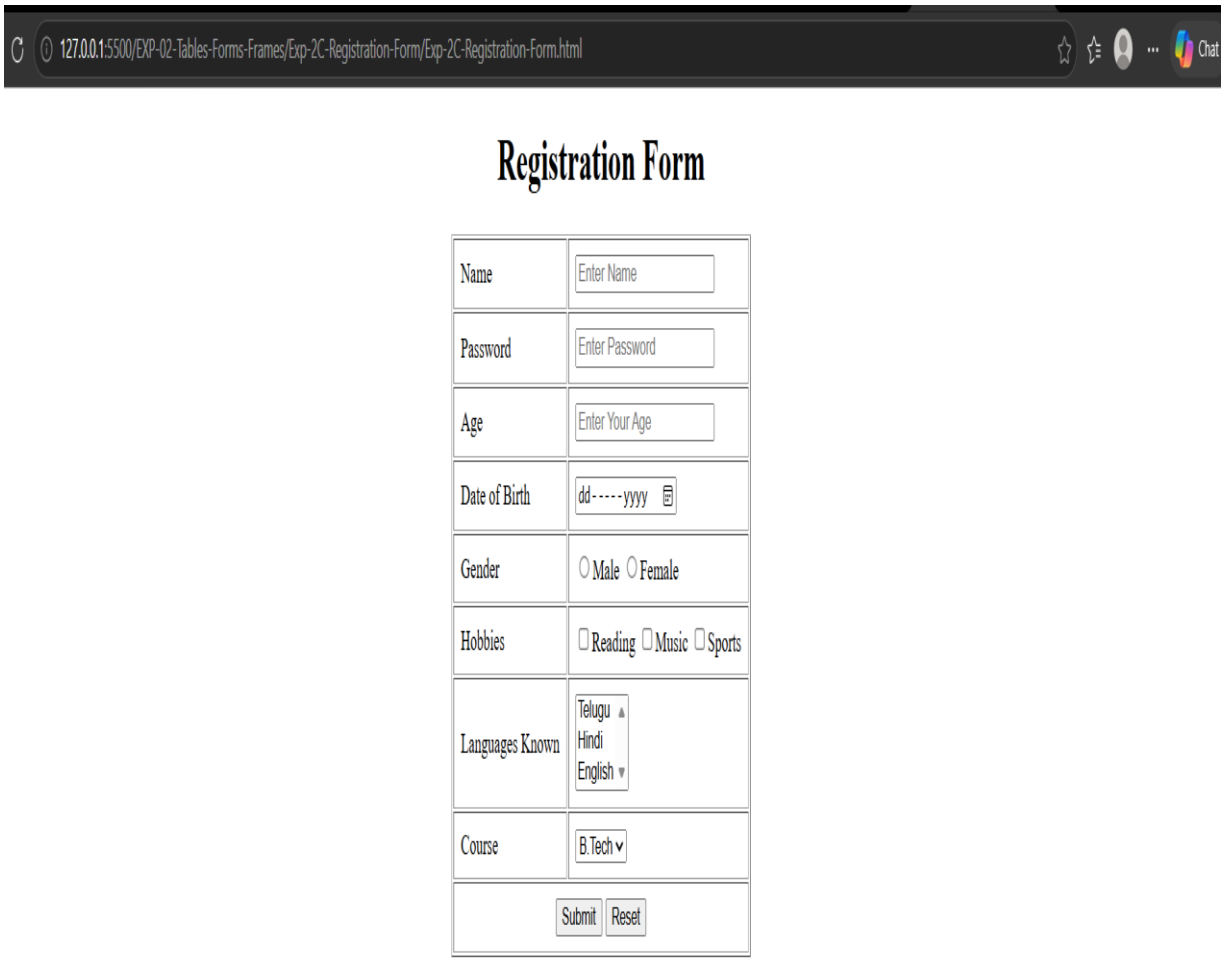
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Registration Form</title>
  </head>
  <body>
    <h1 align="center">Registration Form</h1>
    <form>
      <table border="1" cellpadding="8" align="center">
```

```
<tr>
  <td>Name</td>
  <td><input type="text" placeholder="Enter Name" /></td>
</tr>
<tr>
  <td>Password</td>
  <td>
    <input type="password" placeholder="Enter Password" />
  </td>
</tr>
<tr>
  <td>Age</td>
  <td><input type="number" placeholder="Enter Your Age" /></td>
</tr>
<tr>
  <td>Date of Birth</td>
  <td><input type="date" /></td>
</tr>
<tr>
  <td>Gender</td>
  <td>
    <input type="radio" value="male" />Male
    <input type="radio" value="female" />Female
  </td>
</tr>
<tr>
  <td>Hobbies</td>
  <td>
    <input type="checkbox" value="reading" />Reading
    <input type="checkbox" value="music" />Music
    <input type="checkbox" value="sports" />Sports
  </td>
</tr>
<tr>
```

```
        <td>Languages Known</td>
        <td>
          <select name="languages" size="3" multiple>
            <option>Telugu</option>
            <option>Hindi</option>
            <option>English</option>
          </select>
        </td>
      </tr>
      <tr>
        <td>Course</td>
        <td>
          <select name="course">
            <option>B.Tech</option>
            <option>B.Sc</option>
            <option>BCA</option>
          </select>
        </td>
      </tr>
      <tr>
        <td colspan="2" align="center">
          <input type="submit" value="Submit" />
          <input type="reset" value="Reset" />
        </td>
      </tr>
    </table>
  </form>
 </body>
</html>
```

Output:


127.0.0.1:5500/EXP-02-Tables-Forms-Frames/Exp-2C-Registration-Form/Exp-2C-Registration-Form.html
Chat
Registration Form
Name
Enter Name
Password
Enter Password
Age
Enter Your Age
Date of Birth
dd-----yyyy
Gender
Male
Female
Hobbies
Reading
Music
Sports
Languages Known
Telugu
Hindi
English
Course
B.Tech
Submit
Reset

Experiment: 2d **Date:**

Frames

Create a HTML page with frames that divide the page into three parts: one showing an image, one showing a paragraph and one showing a hyperlink. Use the "no frame" (noresize) attribute.

Aim:

To create a HTML page using frames that divides the browser window into three parts displaying an image, a paragraph and a hyperlink, using the noresize attribute.

Description:

<frameset>: Divides the browser window into rows or columns. Here rows="33%,33%,34%" divides the window into three horizontal parts.

<frame>: Defines one part of the frameset. Its src attribute gives the HTML page to be displayed in that part.

noresize: Prevents the user from changing the size of the frame by dragging its border.

Pages used: image.html (image), paragraph.html (paragraph) and link.html (hyperlink) are loaded into the three frames.

Program:

Exp-2D-Frames.html (main page):

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Frames</title>
</head>
<frameset rows="33%,33%,34%">
<frame src="image.html" noresize="noresize" frameborder="0">
<frame src="paragraph.html" noresize="noresize" frameborder="0">
<frame src="link.html" noresize="noresize" frameborder="0">
</frameset>
</html>
```

image.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Image Frame</title>
</head>
<body>
<h3>Image Frame</h3>
<img src="pic.jpg" alt="Picture" width="150" height="100">
</body>
</html>
```

paragraph.html:

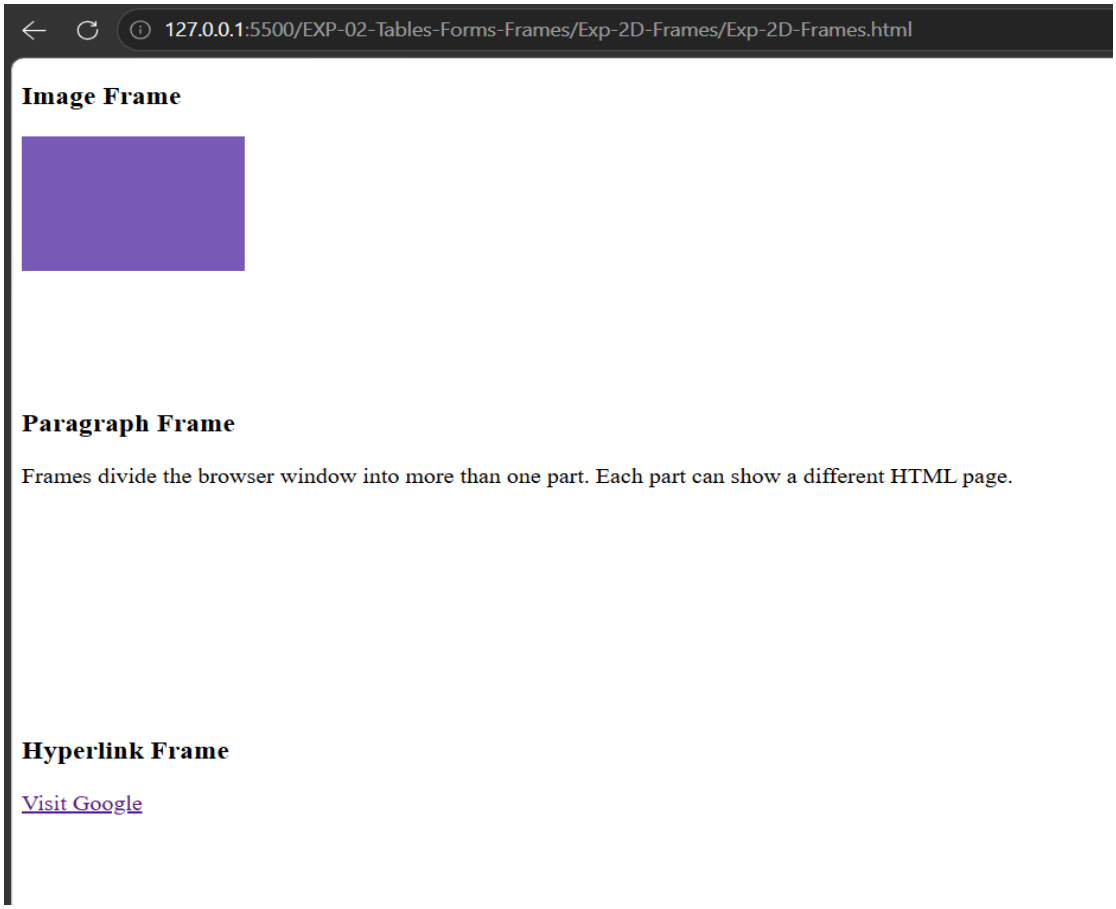
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Paragraph Frame</title>
</head>
<body>
<h3>Paragraph Frame</h3>
<p>Frames divide the browser window into more than one part. Each part can show a
different HTML page.</p>
</body>
</html>
```

link.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Hyperlink Frame</title>
```

```
</head>
<body>
<h3>Hyperlink Frame</h3>
<a href="https://www.google.com" target="_blank">Visit Google</a>
</body>
</html>
```

Output:



Experiment: 3a **Date:**

HTML5 Semantic Tags

Write a HTML program using the tags <footer>, <header>, <main>, <nav>, <section>, and .

Aim:

To write a HTML5 program that uses the semantic tags <header>, <nav>, <main>, <section>, <footer> and the generic tags and .

Description:

<header>: Holds the introductory content or heading of a page.

<nav>: Holds the navigation links of the page.

<main>: Holds the main content of the page.

<section>: Groups related content into a section of the page.

<footer>: Holds the footer information such as copyright details.

** and :** is a block-level container used to group elements; is an inline container used to mark a part of the text.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>HTML5 Semantic Tags</title>
  </head>
  <body>
    <header>
      <h1>My Website</h1>
    </header>
    <nav>
      <a href="#home">Home</a> | <a href="#about">About</a> |
      <a href="#contact">Contact</a>
    </nav>
```

```
    <main>
      <section id="home">
        <h2>Home Section</h2>
        <p>Welcome to my <span>simple</span> website.</p>
      </section>
      <section id="about">
        <h2>About Section</h2>
        <div>
          <p>This is a div inside the about section.</p>
        </div>
      </section>
    </main>
    <footer>
      <p>Copyright 2026. All rights reserved.</p>
    </footer>
  </body>
</html>
```

Output:



Experiment: 3b **Date:**

Audio and Video

Write a HTML program to embed audio and video in a web page.

Aim:

To write a HTML5 program to embed audio and video in a web page using the <audio> and <video> tags.

Description:

<audio>: Embeds a sound file in the page. The controls attribute shows the play, pause and volume controls.

<video>: Embeds a video in the page. The width and height attributes set the size and controls shows the player controls.

<source>: Specifies the media file (src) and its type (for example audio/mpeg, video/mp4). The text inside the tag is shown if the browser does not support the element.

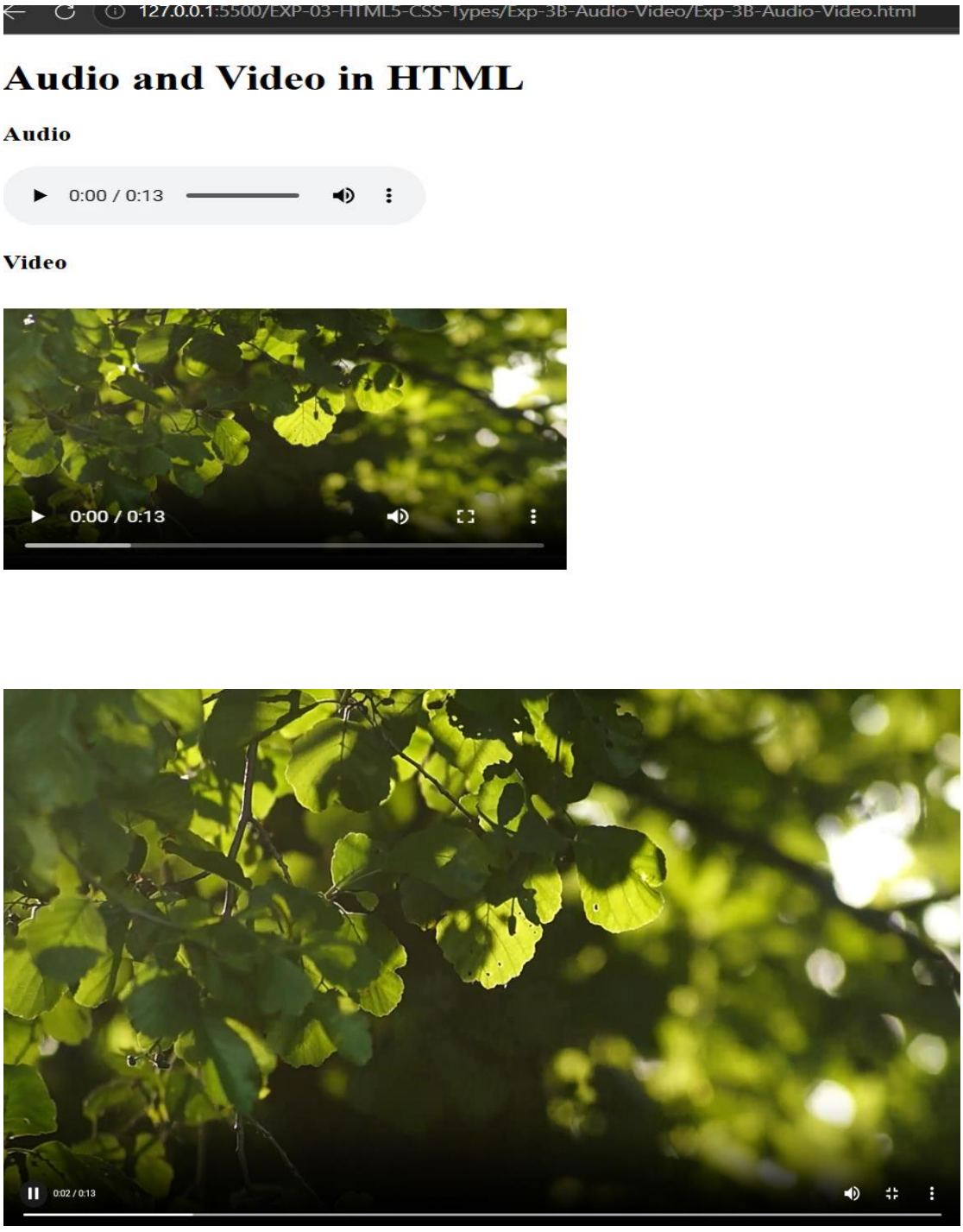
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Audio and Video</title>
  </head>
  <body>
    <h1>Audio and Video in HTML</h1>
    <h3>Audio</h3>
    <audio controls>
      <source src="sample.mp3" type="audio/mpeg" />
      Your browser does not support audio.
    </audio>

    <h3>Video</h3>
```

```
    <video width="400" height="250" controls>
      <source src="sample.mp4" type="video/mp4" />
      Your browser does not support video.
    </video>
  </body>
</html>
```

Output:



Experiment: 3c **Date:**

Types of CSS

Apply inline, internal and external styles to a web page and identify the selector, property and value in each.

Aim:

To apply inline, internal and external CSS to a web page and to identify the selector, property and value.

Description:

Inline CSS: Style is written in the style attribute of an individual HTML element and affects only that element.

Internal CSS: Style rules are written inside a <style> tag in the <head> section and apply to the whole page.

External CSS: Style rules are written in a separate .css file which is linked to the page using the <link> tag; one file can style many pages.

Selector, property, value: In h2 { color: blue; } the selector is h2 (the element to style), the property is color (what to change) and the value is blue (the setting given to the property).

Program:

Exp-3C-Types-of-CSS.html:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Types of CSS</title>
    <link rel="stylesheet" href="style.css" />
    <style>
      h2 {
        color: blue;
      }
    </style>
```

```
  </head>
  <body>
    <h1>Types of CSS</h1>
    <h2>Internal CSS: this heading is blue</h2>

    <p class="External">
      External CSS: this paragraph is green with a yellow background
    </p>

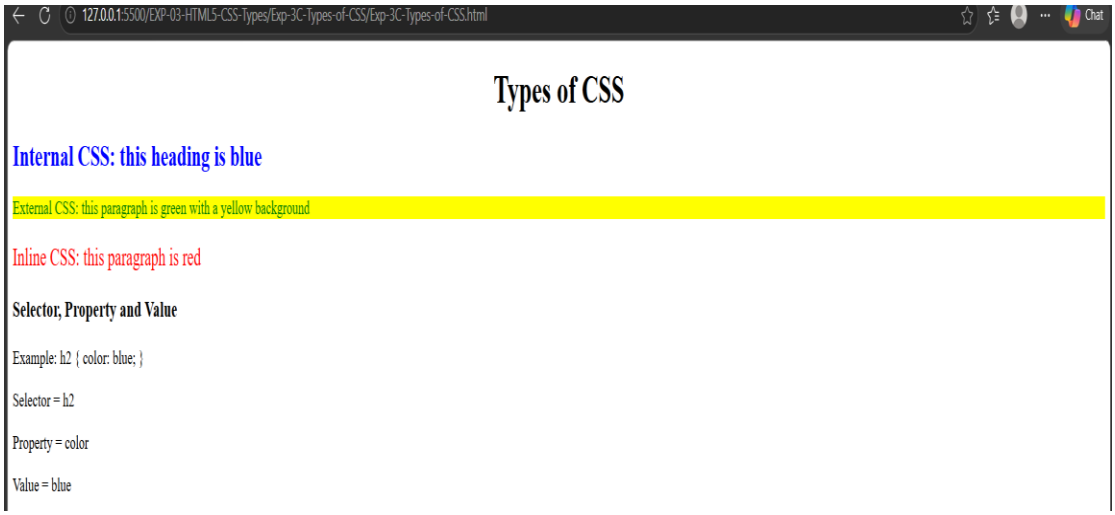
    <p style="color: red; font-size: 20px">Inline CSS: this paragraph is red</p>

    <h3>Selector, Property and Value</h3>
    <p>Example: h2 { color: blue; }</p>
    <p>Selector = h2</p>
    <p>Property = color</p>
    <p>Value = blue</p>
  </body>
</html>
```

style.css (external style sheet):

```
.External {
  color: green;
  background-color: yellow;
}
h1 {
  text-align: center;
}
```

Output:


127.0.0.1:5500/EXP-03-HTML5-CSS-Types/Exp-3C-Types-of-CSS/Exp-3C-Types-of-CSS.html
Chat
Types of CSS
Internal CSS: this heading is blue
External CSS: this paragraph is green with a yellow background
Inline CSS: this paragraph is red
Selector, Property and Value
Example: h2 { color: blue; }
Selector = h2
Property = color
Value = blue

Experiment: 4a **Date:**

Selector Forms

Apply different selector forms: (i) Simple selector (element, id, class, group, universal), (ii) Combinator selector (descendant, child, adjacent sibling, general sibling), (iii) Pseudo-class selector, (iv) Pseudo-element selector, (v) Attribute selector.

(i) Simple Selector

Aim:

To apply the simple selectors (element, id, class, group and universal selector) in CSS.

Description:

Element selector: Selects all elements of a given tag name, e.g. h1.

ID selector: Selects the single element with a given id; written with #, e.g. #special.

Class selector: Selects all elements with a given class; written with a dot, e.g. .note.

Group selector: Applies the same style to several selectors separated by commas, e.g. h3, h4.

Universal selector: Selects every element on the page; written with *.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Simple Selectors</title>
    <style>
      h1 {
        color: blue;
      }
      #special {
        color: red;
      }
      .note {
```

```
      background-color: yellow;
    }
    h3,
    h4 {
      color: green;
    }
    * {
      font-family: Arial;
      background-color: rgb(180, 231, 231);
    }
  </style>
 </head>
 <body>
  <h1>Element Selector</h1>
  <p id="special">ID Selector</p>
  <p class="note">Class Selector</p>
  <h3>Group Selector h3</h3>
  <h4>Group Selector h4</h4>
  <p>Universal Selector applies Arial font to everything</p>
 </body>
</html>
```

Output:



(ii) Combinator Selector

Aim:

To apply the combinator selectors (descendant, child, adjacent sibling and general sibling) in CSS.

Description:

Descendant selector (div p): Selects all <p> elements inside a , at any level.

Child selector (div > span): Selects only the elements that are direct children of a .

Adjacent sibling selector (h2 + p): Selects the <p> element placed immediately after an <h2>.

General sibling selector (h2 ~ p): Selects all <p> elements that follow an <h2> and share the same parent.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Combinator Selectors</title>
    <style>
      div p {
        color: blue;
      }
      div > span {
        color: red;
      }
      h2 + p {
        background-color: yellow;
      }
      h2 ~ p {
        font-weight: bold;
      }
    </style>
```

```
  </head>
  <body>
    <div>
      <p>Descendant selector paragraph</p>
      <span>Child selector span</span>
      <section>
        <span>This span is not a direct child</span>
      </section>
    </div>
    <h2>Heading</h2>
    <p>Adjacent sibling: right after the heading</p>
    <p>General sibling: after the heading</p>
    <p>General sibling: after the heading</p>
  </body>
</html>
```

Output:



Descendant selector paragraph

Child selector span
This span is not a direct child

Heading

Adjacent sibling: right after the heading

General sibling: after the heading

General sibling: after the heading

(iii) Pseudo-class Selector

Aim:

To apply pseudo-class selectors in CSS.

Description:

Pseudo-class: A keyword added to a selector with a colon (:) that styles an element in a special state.

:hover / :active: Styles a button when the mouse is over it / while it is being clicked.

:focus: Styles an input box while it is selected for typing.

:first-child: Styles an element that is the first child of its parent (the first list item).

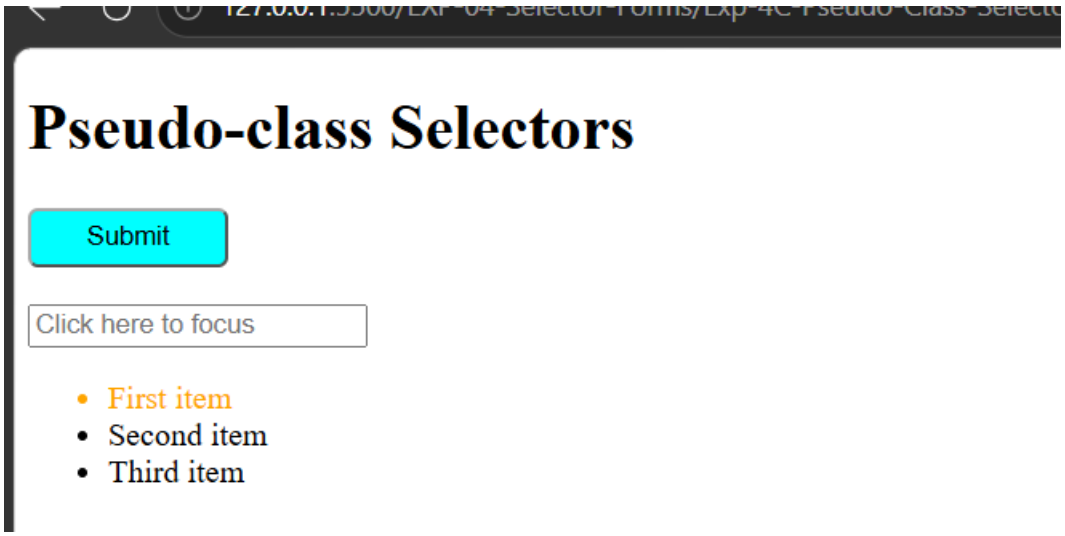
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Pseudo-class Selectors</title>
    <style>
      button {
        background-color: aqua;
        width: 100px;
        height: 30px;
        border-radius: 5px;
        border-color: black;
      }
      button:hover {
        background-color: rgb(106, 248, 106);
        color: red;
      }
      button:active {
        background-color: white;
      }
      input:focus {
        background-color: lightyellow;
```

```
    }
    li:first-child {
      color: orange;
    }
  </style>
</head>
<body>
  <h1>Pseudo-class Selectors</h1>
  <button>Submit</button>
  <br /><br />
  <input type="text" placeholder="Click here to focus" />

  <ul>
    <li>First item</li>
    <li>Second item</li>
    <li>Third item</li>
  </ul>
</body>
</html>
```

Output:



(iv) Pseudo-element Selector

Aim:

To apply pseudo-element selectors in CSS.

Description:

Pseudo-element: A keyword added to a selector with double colon (::) that styles a specific part of an element.

::first-line / ::first-letter: Style the first line / the first letter of a paragraph.

::before / ::after: Insert generated content before / after the content of an element, using the content property.

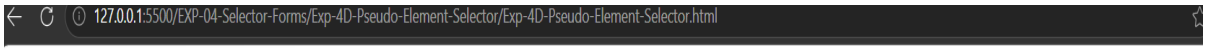
::selection: Styles the part of the text selected by the user.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Pseudo-element Selectors</title>
    <style>
      p::first-line {
        color: blue;
      }
      p::first-letter {
        font-size: 30px;
        color: red;
      }
      h2::before {
        content: "Start - ";
        color: green;
      }
      h2::after {
        content: " - End";
        color: green;
      }
```

```
    ::selection {
      background-color: yellow;
    }
  </style>
 </head>
 <body>
  <h1>Pseudo-element Selectors</h1>
  <p>
    This is a paragraph used to show the first line and the first letter
    styles. It is written with a little more text so that it goes onto a
    second line in the browser window and we can see the effect clearly.
  </p>
  <h2>Heading</h2>
 </body>
</html>
```

Output:



Pseudo-element Selectors

This is a paragraph used to show the first line and the first letter styles. It is written with a little more text so that it goes onto a second line in the browser window and we can see the effect clearly.

Start - Heading - End

(v) Attribute Selector

Aim:

To apply attribute selectors in CSS.

Description:

Attribute selector: Selects elements based on an attribute or attribute value, written inside square brackets.

a[target]: Selects all links that have a target attribute.

input[type="text"]: Selects only the input elements whose type attribute is text.

Program:

```
<!doctype html>
<html>
  <head>
    <title>Attribute Selectors</title>
    <style>
      a[target] {
        color: blue;
      }

      input[type="text"] {
        background-color: lightblue;
      }
    </style>
  </head>
  <body>
    <h1>Attribute Selectors</h1>

    <a href="https://google.com" target="_blank">Google</a>
    <br /><br />

    <input type="text" placeholder="Enter name" />
  </body>
</html>
```

Output:


127.0.0.1:5500/EXP-04-Selector-Forms/Exp-4E-Attribute-Selector/Exp-4E-Attribute-Selector.html
Attribute Selectors
Google
Enter name

Experiment: 5a **Date:**

Colors in CSS

Write a program to reference colors in CSS in different ways.

Aim:

To write a HTML program to reference colors in CSS in different ways (color name, hexadecimal code, RGB and RGBA values).

Description:

Color name: A predefined name such as red.

Hexadecimal code: A # followed by six hex digits for red, green and blue, e.g. #0000ff.

RGB value: rgb(red, green, blue) with each value from 0 to 255, e.g. rgb(0, 128, 0).

RGBA value: Same as RGB with an extra alpha value (0 to 1) that sets transparency, e.g. rgba(255, 0, 0, 0.3).

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>CSS Colors</title>
    <style>
      .name {
        color: red;
      }
      .hex {
        color: #0000ff;
      }
      .rgb {
        color: rgb(0, 128, 0);
      }
      .rgba {
        background-color: rgba(255, 0, 0, 0.3);
```

```
    }
  </style>
 </head>
 <body>
  <h1>Referencing Colors in CSS</h1>
  <p class="name">Color name: red</p>
  <p class="hex">Hex code: #0000ff</p>
  <p class="rgb">RGB value: rgb(0, 128, 0)</p>
  <p class="rgba">RGBA value with transparency: rgba(255, 0, 0, 0.3)</p>
 </body>
</html>
```

Output:



Experiment: 5b **Date:**

Background Image

Write a program to place a background image halfway down the page, tiled (repeated) horizontally, that remains in place when the page is scrolled.

Aim:

To write a HTML program with a background image that is positioned halfway down the page, tiled horizontally and remains fixed when the page is scrolled.

Description:

background-image: Sets the image used as the background of the element.

background-repeat: repeat-x: Repeats the image only in the horizontal direction.

background-position: left 50%: Places the image at the left edge and halfway (50%) down the page.

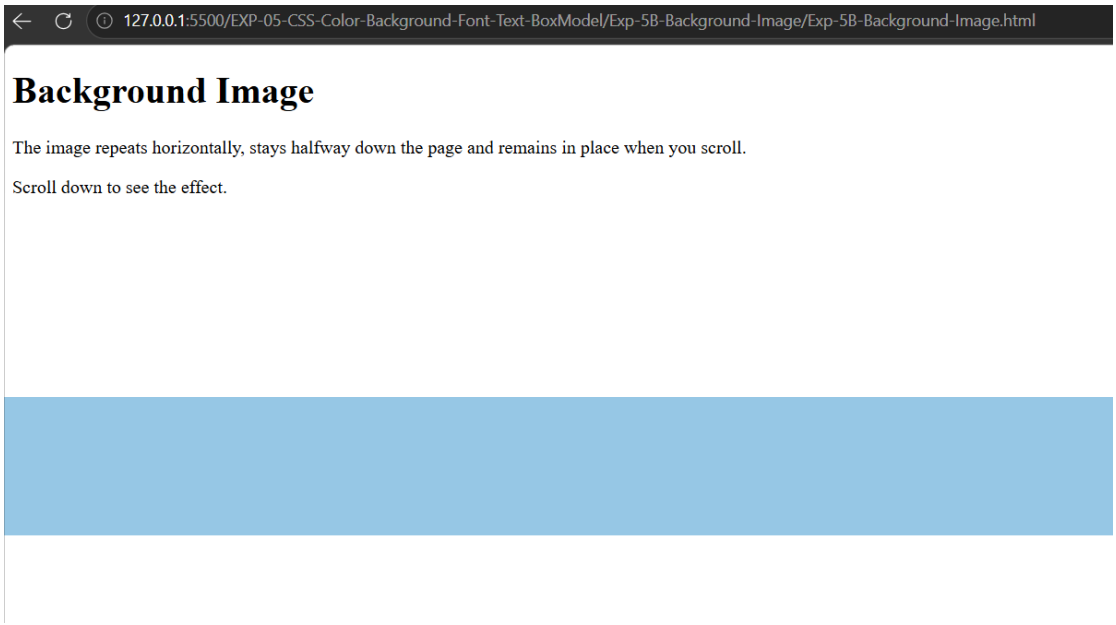
background-attachment: fixed: Keeps the image fixed in the window so it does not move when the page is scrolled.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Background Image</title>
    <style>
      body {
        background-image: url("bg.jpg");
        background-repeat: repeat-x;
        background-position: left 50%;
        background-attachment: fixed;
      }
    </style>
  </head>
  <body>
```

```
    <h1>Background Image</h1>
    <p>
      The image repeats horizontally, stays halfway down the page and remains in
      place when you scroll.
    </p>
    <p>Scroll down to see the effect.</p>
    <br /><br /><br /><br /><br /><br /><br /><br /><br /><br /><br /><br
/><br /><br /><br /><br /><br /><br /><br />
    <br /><br /><br /><br /><br /><br /><br /><br /><br /><br /><br /><br
/><br /><br /><br /><br /><br /><br /><br />
    <p>End of the page</p>
  </body>
</html>
```

Output:



Experiment: 5c **Date:**

Font and Text Properties

Use the CSS properties: (i) font-size, (ii) font-weight, (iii) font-style, (iv) text-decoration, (v) text-transform, (vi) text-align.

Aim:

To write a HTML program that uses the CSS properties font-size, font-weight, font-style, text-decoration, text-transform and text-align.

Description:

font-size: Sets the size of the text, e.g. 24px.

font-weight: Sets the thickness of the text, e.g. bold.

font-style: Sets the style of the text, e.g. italic.

text-decoration: Adds a line to the text, e.g. underline or line-through.

text-transform: Changes the case of the text: uppercase, lowercase or capitalize.

text-align: Aligns the text horizontally: left, center or right.

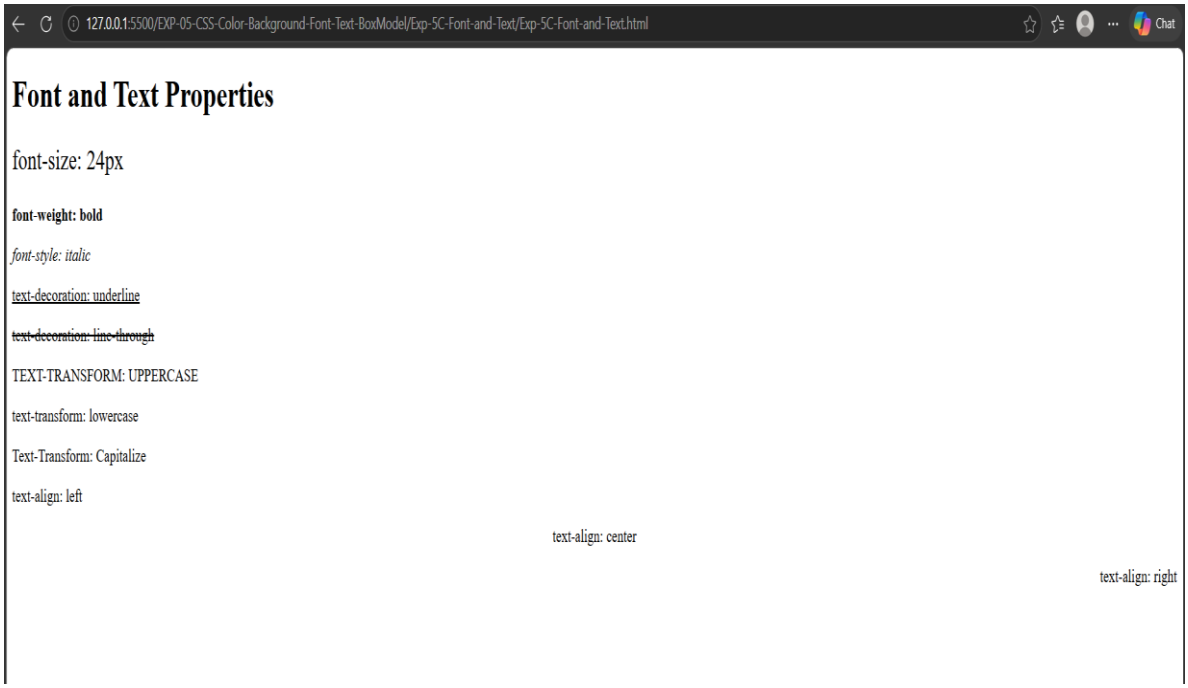
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Font and Text Properties</title>
    <style>
      .size {
        font-size: 24px;
      }
      .weight {
        font-weight: bold;
      }
      .style {
        font-style: italic;
      }
```

```
    .underline {
      text-decoration: underline;
    }
    .linethrough {
      text-decoration: line-through;
    }
    .upper {
      text-transform: uppercase;
    }
    .lower {
      text-transform: lowercase;
    }
    .capital {
      text-transform: capitalize;
    }
    .left {
      text-align: left;
    }
    .center {
      text-align: center;
    }
    .right {
      text-align: right;
    }
  </style>
</head>
<body>
  <h1>Font and Text Properties</h1>
  <p class="size">font-size: 24px</p>
  <p class="weight">font-weight: bold</p>
  <p class="style">font-style: italic</p>
  <p class="underline">text-decoration: underline</p>
  <p class="linethrough">text-decoration: line-through</p>
  <p class="upper">text-transform: uppercase</p>
```

```
    <p class="lower">TEXT-TRANSFORM: LOWERCASE</p>
    <p class="capital">text-transform: capitalize</p>
    <p class="left">text-align: left</p>
    <p class="center">text-align: center</p>
    <p class="right">text-align: right</p>
  </body>
</html>
```

Output:



Experiment: 5d **Date:**

CSS Box Model

Write a program to demonstrate the CSS Box Model: (i) Content, (ii) Border, (iii) Margin, (iv) Padding.

Aim:

To write a HTML program to demonstrate the CSS box model using content, border, margin and padding.

Description:

Content: The actual text or image of the element, whose size is set by width and height.

Padding: The space between the content and the border.

Border: The line that surrounds the padding and the content.

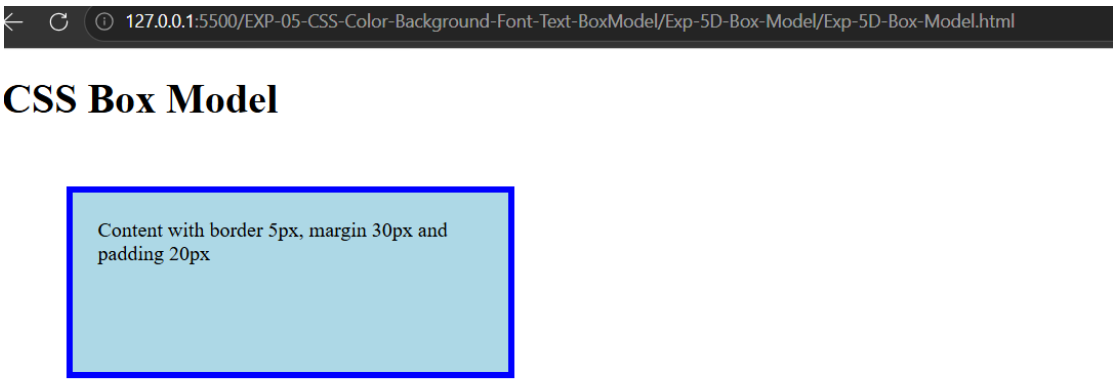
Margin: The space outside the border that separates the element from other elements.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>CSS Box Model</title>
    <style>
      .box {
        width: 300px;
        height: 100px;
        background-color: lightblue;
        border: 5px solid blue;
        margin: 50px;
        padding: 20px;
      }
    </style>
  </head>
  <body>
```

```
    <h1>CSS Box Model</h1>
    <div class="box">Content with border 5px, margin 30px and padding 20px</div>
  </body>
</html>
```

Output:



Experiment: 6a **Date:**

Internal and External JavaScript

Write a program to embed internal and external JavaScript in a HTML page.

Aim:

To write a HTML program to embed internal and external JavaScript in a web page.

Description:

Internal JavaScript: JavaScript code written inside a <script> tag within the HTML page itself.

External JavaScript: JavaScript code written in a separate .js file and linked using <script src="file.js"></script>. The same file can be reused in many pages.

document.write(): Writes the given text or HTML into the web page.

Program:

Exp-6A-Embed-JavaScript.html:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Embed JavaScript</title>
  </head>
  <body>
    <h1>Internal and External JavaScript</h1>
    <script>
      document.write("<p>This message is from Internal JavaScript</p>");
    </script>
    <script src="script.js"></script>
  </body>
</html>
```

script.js (external JavaScript file):

```
document.write("<p>This message is from External JavaScript</p>");
```

```
console.log("This is external js");
```

Output:



Experiment: 6b **Date:**

JavaScript Output Methods

Write a program to demonstrate the JavaScript output methods.

Aim:

To write a JavaScript program to demonstrate the output methods document.write(), innerHTML, console.log() and alert().

Description:

document.write(): Writes the output directly into the web page.

innerHTML: Places the output inside an HTML element selected with document.getElementById().

console.log(): Writes the output to the browser console (seen with F12 → Console).

alert(): Displays the output in a pop-up alert box.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>JavaScript Output Methods</title>
  </head>
  <body>
    <h1>JavaScript Output Methods</h1>
    <p id="demo"></p>
    <script>
      document.write("<p>This is  document.write()</p>");
      document.getElementById("demo").innerHTML = "Output using innerHTML";
      console.log("Output using console.log() - see the browser console");
      alert("Output using alert()");
    </script>
  </body>
</html>
```

Output:


127.0.0.1:5500/EXP-06-JS-Internal-External-IO-TypeConversion/Exp-6B-Output-Methods/Exp-6B-Output-Methods.html
Chat
127.0.0.1:5500 says
Output using alert()
OK

Experiment: 6c **Date:**

JavaScript Input Methods

Write a program to demonstrate the JavaScript input methods.

Aim:

To write a JavaScript program to demonstrate the input methods prompt() and reading values from an HTML text box.

Description:

prompt(): Displays a dialog box asking the user to enter a value; the entered value is returned as a string.

Text box input: The value typed in an <input> element is read using document.getElementById("id").value, here when the button is clicked (onclick event).

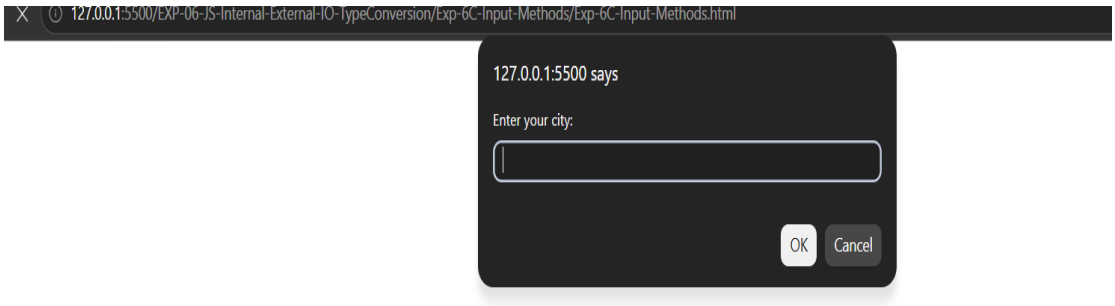
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>JavaScript Input Methods</title>
  </head>
  <body>
    <h1>JavaScript Input Methods</h1>
    <input type="text" id="name" placeholder="Enter your name" />
    <button onclick="showName()">Show Name</button>
    <p id="result"></p>
    <script>
      var city = prompt("Enter your city:");
      document.write("<p>City: " + city + "</p>");

      function showName() {
        var n = document.getElementById("name").value;
        document.getElementById("result").innerHTML = "Hello " + n;
```

```
    }
  </script>
 </body>
</html>
```

Output:





Experiment: 6d **Date:**

Voter Eligibility

Write a program to prompt for a name and age, display them in a table and also note whether the person is eligible to vote.

Aim:

To write a JavaScript program that accepts a name and age using prompt(), displays them in a table and shows whether the person is eligible to vote.

Description:

prompt(): Reads the name and age from the user as strings.

Type conversion: Number() converts the age entered as a string into a number so that it can be compared.

Condition: If age >= 18 the person is eligible to vote, otherwise not eligible.

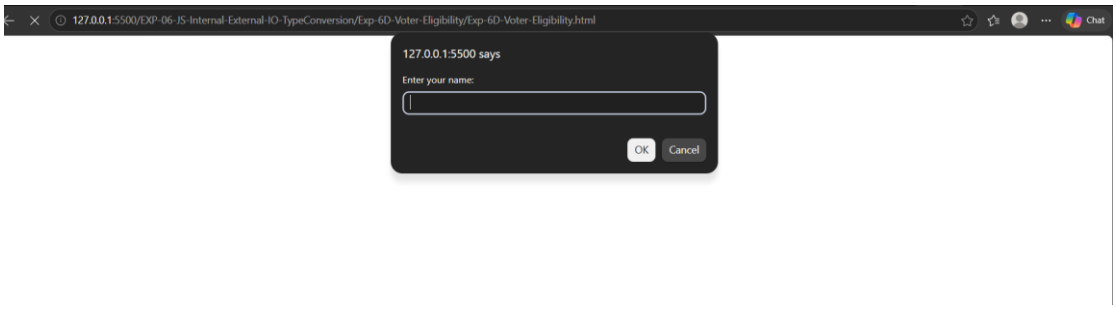
Output: document.write() is used to build an HTML table showing the name, age and voter status.

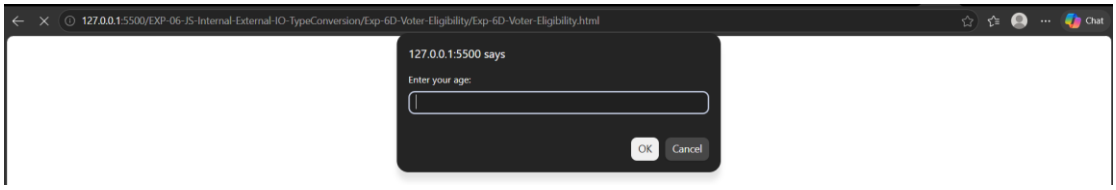
Program:

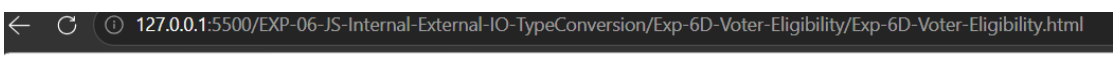
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Voter Eligibility</title>
  </head>
  <body>
    <h1>Voter Eligibility Check</h1>
    <script>
      var name = prompt("Enter your name:");
      var age = Number(prompt("Enter your age:"));
      var status;
      if (age >= 18) {
        status = "Eligible to vote";
      } else {
```

```
      status = "Not eligible to vote";
    }
    document.write("<table border='1' cellpadding='8'>");
    document.write("<tr><th>Name</th><th>Age</th><th>Voter Status</th></tr>");
    document.write(
      "<tr><td>" +
        name +
        "</td><td>" +
        age +
        "</td><td>" +
        status +
        "</td></tr>",   );
    document.write("</table>");
  </script>
 </body>
</html>
```

Output:







Voter Eligibility Check

Name	Age	Voter Status
kumar	20	Eligible to vote

Experiment: 7a **Date:**

Document Object

Write a program to demonstrate the properties and methods of the Document object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the Document object.

Description:

Document object: Represents the web page loaded in the browser and is the entry point for accessing and changing its content.

Properties used: document.title (page title), document.bgColor (background color) and document.URL (address of the page).

Methods used: document.write() writes into the page and document.getElementById() accesses an element so that its innerHTML can be changed.

Program:

```
<!doctype html>
<html>
  <head>
    <title>Document Object</title>
  </head>
  <body>
    <h2 id="heading">Welcome</h2>
    <p id="demo">Hello World</p>

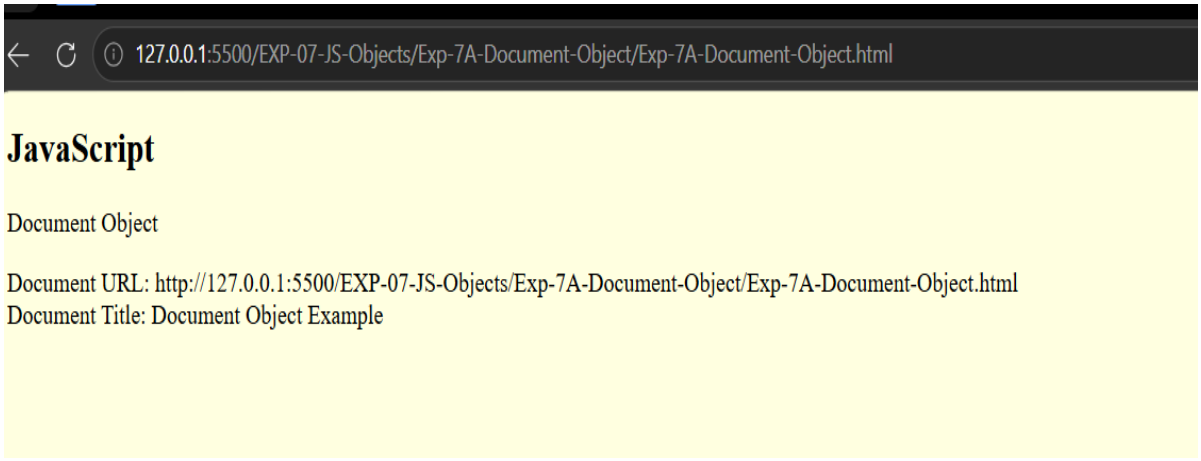
    <script>
      // Document object properties
      document.title = "Document Object Example";
      document.bgColor = "lightyellow";

      // Document object methods
```

```
    document.write("Document URL: " + document.URL + "<br>");
    document.write("Document Title: " + document.title + "<br>");

    // Change HTML content
    document.getElementById("heading").innerHTML = "JavaScript";
    document.getElementById("demo").innerHTML = "Document Object";
  </script>
 </body>
</html>
```

Output:



Experiment: 7b **Date:**

Window Object

Write a program to demonstrate the properties and methods of the Window object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the Window object.

Description:

Window object: Represents the browser window; all global objects, functions and variables belong to it.

Properties used: window.innerWidth and window.innerHeight give the width and height of the browser window's content area.

Methods used: window.alert() shows a message box, window.confirm() shows an OK/Cancel box and window.open() opens a new window or tab.

Program:

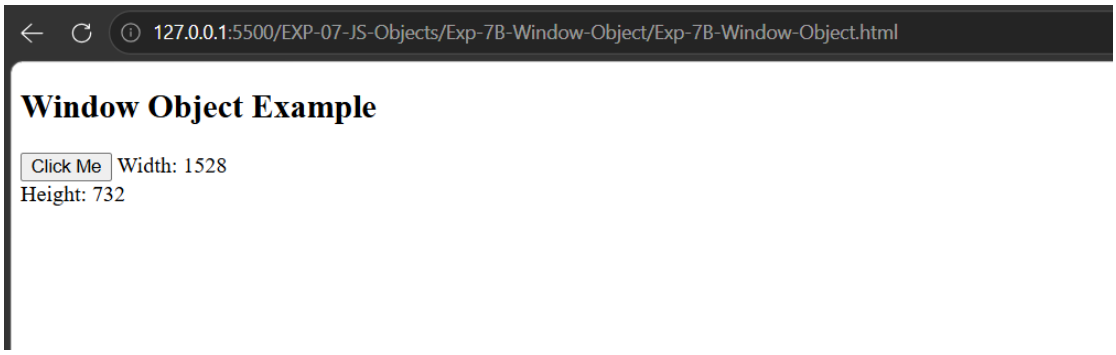
```
<!doctype html>
<html>
  <head>
    <title>Window Object</title>
  </head>
  <body>
    <h2>Window Object Example</h2>

    <button onclick="showInfo()">Click Me</button>

    <script>
      // Window object properties
      document.write("Width: " + window.innerWidth + "<br>");
      document.write("Height: " + window.innerHeight + "<br>");
```

```
    // Window object methods
    function showInfo() {
      alert("Welcome to JavaScript!");
      window.confirm("Are you sure?");
      window.open("https://www.google.com");
    }
  </script>
 </body>
</html>
```

Output:



Experiment: 7c **Date:**

Array Object

Write a program to demonstrate the properties and methods of the Array object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the Array object.

Description:

Array object: Stores multiple values in a single variable.

Property used: length returns the number of elements in the array.

Methods used: push() adds an element at the end, pop() removes the last element and sort() arranges the elements in alphabetical order.

Program:

```
<!doctype html>
<html>
  <head>
    <title>Array Object</title>
  </head>
  <body>
    <h2>JavaScript Array Object</h2>

    <script>
      // Creating an array
      let fruits = ["Apple", "Banana", "Mango"];

      // Array property
      document.write("Length: " + fruits.length + "<br>");

      // Array methods
      fruits.push("Orange");
      document.write("After Push: " + fruits + "<br>");
```

```
    fruits.pop();
    document.write("After Pop: " + fruits + "<br>");

    fruits.sort();
    document.write("After Sort: " + fruits + "<br>");
  </script>
 </body>
</html>
```

Output:



Experiment: 7d **Date:**

Math Object

Write a program to demonstrate the properties and methods of the Math object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the Math object.

Description:

Math object: A built-in object that provides constants and functions for mathematical tasks.

Properties used: Math.PI (value of π) and Math.E (Euler's number).

Methods used: Math.sqrt() (square root), Math.pow() (power), Math.round() (nearest integer) and Math.max() (largest value).

Program:

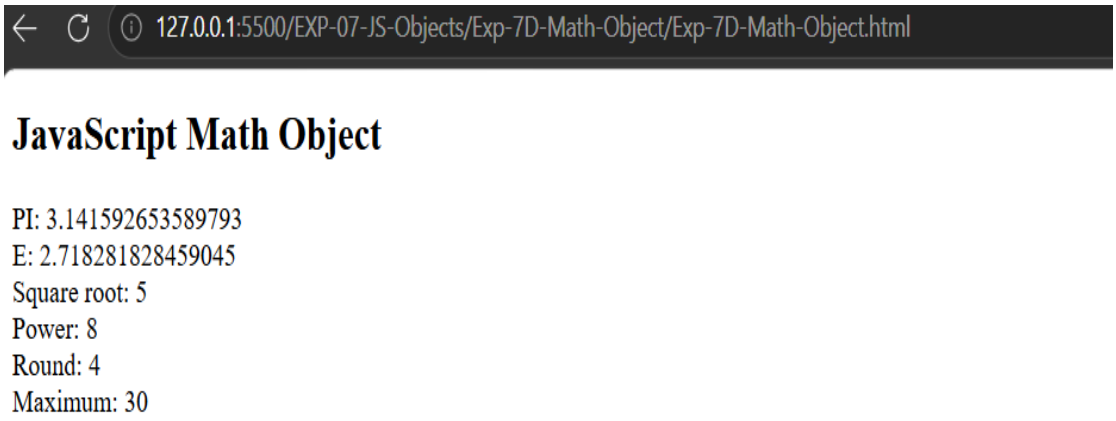
```
<!doctype html>
<html>
  <head>
    <title>Math Object</title>
  </head>
  <body>
    <h2>JavaScript Math Object</h2>

    <script>
      // Math properties
      document.write("PI: " + Math.PI + "<br>");
      document.write("E: " + Math.E + "<br>");

      // Math methods
      document.write("Square root: " + Math.sqrt(25) + "<br>");
      document.write("Power: " + Math.pow(2, 3) + "<br>");
      document.write("Round: " + Math.round(3.6) + "<br>");
```

```
    document.write("Maximum: " + Math.max(10, 20, 30) + "<br>");
  </script>
 </body>
</html>
```

Output:



Experiment: 7e **Date:**

String Object

Write a program to demonstrate the properties and methods of the String object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the String object.

Description:

String object: Used to store and work with text.

Property used: length returns the number of characters in the string.

Methods used: toUpperCase(), toLowerCase(), charAt(), indexOf(), substring(), slice(), replace(), concat(), split() and trim().

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>String Object</title>
  </head>
  <body>
    <h1>String Object</h1>
    <script>
      var str = "Hello World";
      document.write("String: " + str + "<br>");
      document.write("Length: " + str.length + "<br>");
      document.write("toUpperCase(): " + str.toUpperCase() + "<br>");
      document.write("toLowerCase(): " + str.toLowerCase() + "<br>");
      document.write("charAt(1): " + str.charAt(1) + "<br>");
      document.write("indexOf('World'): " + str.indexOf("World") + "<br>");
      document.write("substring(0, 5): " + str.substring(0, 5) + "<br>");
      document.write("slice(6): " + str.slice(6) + "<br>");
```

```
    document.write("replace: " + str.replace("World", "JavaScript") + "<br>");
    document.write("concat: " + str.concat(" Welcome") + "<br>");
    document.write("split: " + str.split(" ") + "<br>");
    document.write("trim: " + "  spaces  ".trim() + "<br>");
  </script>
 </body>
</html>
```

Output:



Experiment: 7f **Date:**

RegExp Object

Write a program to demonstrate the properties and methods of the RegExp (regular expression) object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the RegExp object.

Description:

RegExp object: Describes a pattern of characters that is used to search and match text. Here /hello/i is a pattern with the i (ignore case) flag.

Properties used: source gives the text of the pattern and ignoreCase tells whether the i flag is set.

Methods used: test() returns true or false depending on whether the pattern is found, and exec() returns the matched text.

Program:

```
<!doctype html>
<html>
  <head>
    <title>RegExp Object</title>
  </head>
  <body>
    <h2>JavaScript RegExp Object</h2>

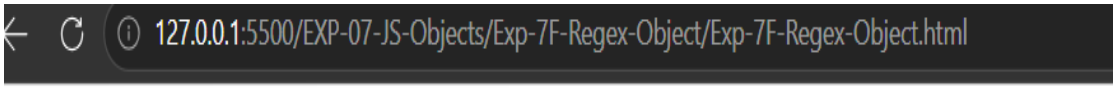
    <script>
      // Creating a regular expression
      let pattern = /hello/i;
      let text = "Hello World";

      // RegExp property
      document.write("Source: " + pattern.source + "<br>");
```

```
    document.write("Ignore Case: " + pattern.ignoreCase + "<br>");

    // RegExp methods
    document.write("Test: " + pattern.test(text) + "<br>");
    document.write("Match: " + pattern.exec(text));
  </script>
 </body>
</html>
```

Output:



JavaScript RegExp Object

Source: hello
Ignore Case: true
Test: true
Match: Hello

Experiment: 7g **Date:**

Date Object

Write a program to demonstrate the properties and methods of the Date object.

Aim:

To write a JavaScript program to demonstrate the properties and methods of the Date object.

Description:

Date object: Used to work with dates and times. new Date() creates an object holding the current date and time.

Methods used: getDate(), getMonth() (0 to 11, so 1 is added), getFullYear(), getHours(), getMinutes() and toDateString().

Program:

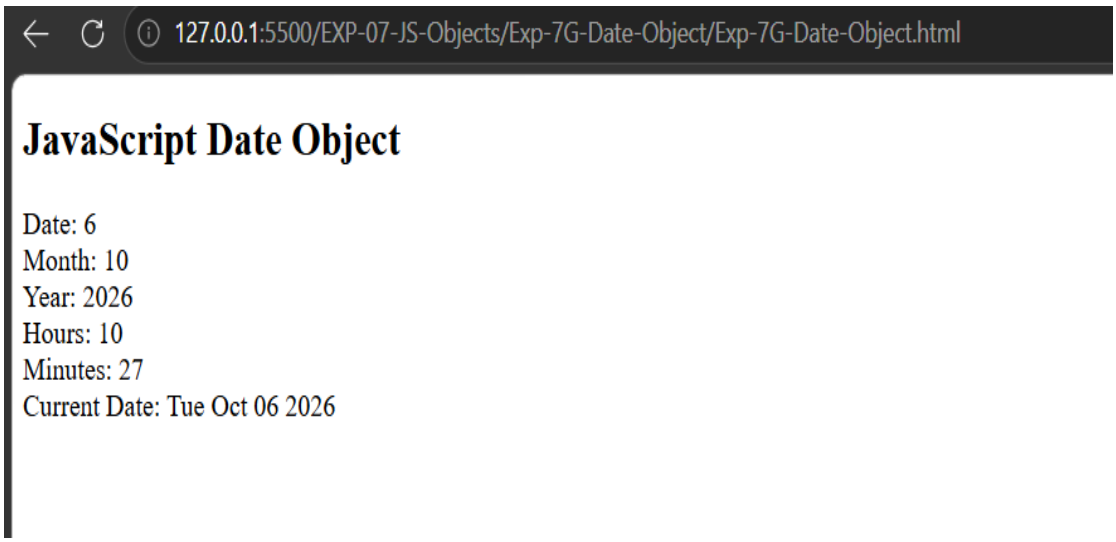
```
<!doctype html>
<html>
  <head>
    <title>Date Object</title>
  </head>
  <body>
    <h2>JavaScript Date Object</h2>

    <script>
      // Creating Date object
      let d = new Date();

      // Date methods
      document.write("Date: " + d.getDate() + "<br>");
      document.write("Month: " + (d.getMonth() + 1) + "<br>");
      document.write("Year: " + d.getFullYear() + "<br>");
      document.write("Hours: " + d.getHours() + "<br>");
      document.write("Minutes: " + d.getMinutes() + "<br>");
```

```
    document.write("Current Date: " + d.toDateString());
  </script>
 </body>
</html>
```

Output:



Experiment: 7h **Date:**

User-defined Object

Write a program to create a user-defined object and demonstrate its properties, methods, constructor and display.

Aim:

To write a JavaScript program to create a user-defined object with properties, a method and a constructor, and to display its data.

Description:

Constructor: A function (here Student) used with the new keyword to create objects of the same kind.

Properties: Variables of the object (name, age), created using the this keyword and accessed with the dot operator, e.g. s1.name.

Method: A function stored in an object (display()) that works on the object's own properties.

new keyword: Creates a new object from the constructor, e.g. let s1 = new Student("Kumar", 20);

Program:

```
<!doctype html>
<html>
  <head>
    <title>User Defined Object</title>
  </head>
  <body>
    <h2>User Defined Object</h2>

    <script>
      // Constructor
      function Student(name, age) {
        this.name = name;
        this.age = age;
```

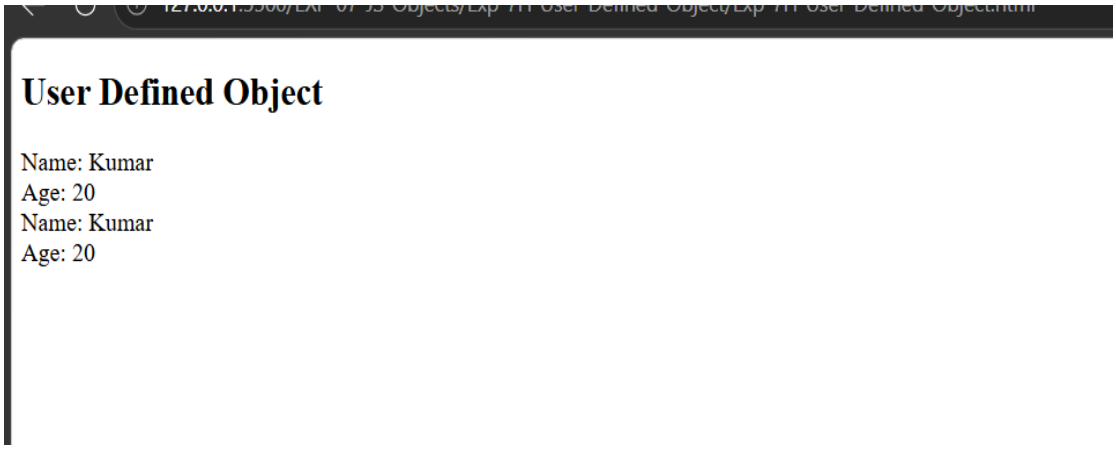
```
    // Method
    this.display = function () {
      document.write("Name: " + this.name + "<br>");
      document.write("Age: " + this.age + "<br>");
    };
  }

  // Creating object
  let s1 = new Student("Kumar", 20);

  // Accessing properties
  document.write("Name: " + s1.name + "<br>");
  document.write("Age: " + s1.age + "<br>");

  // Calling method
  s1.display();
 </script>
</body>
</html>
```

Output:



Experiment: 8a **Date:**

Largest of Three Numbers

Write a program to input three integers and display the largest of them in an HTML dialog box. If all the numbers are equal, display "Equal numbers".

Aim:

To write a JavaScript program to read three integers and display the largest number (or "Equal numbers") in an alert dialog box using if-else statements.

Description:

if … else if … else: A conditional statement that executes one block of code out of several depending on which condition is true.

Logic: If all three numbers are equal, "Equal numbers" is shown; otherwise each number is compared with the other two using >= and the largest is displayed.

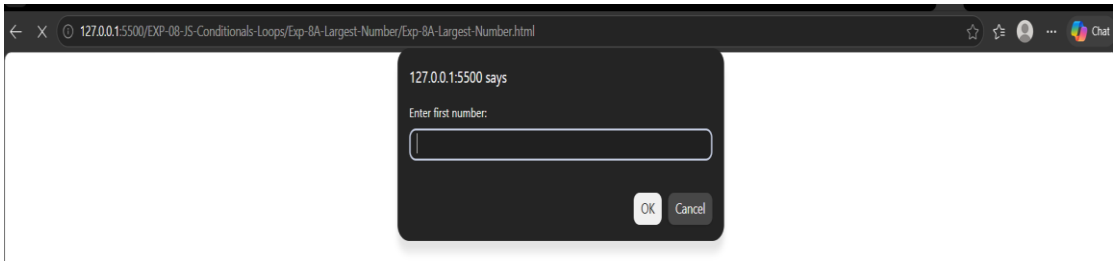
prompt() and alert(): prompt() reads the numbers and alert() displays the result in a dialog box.

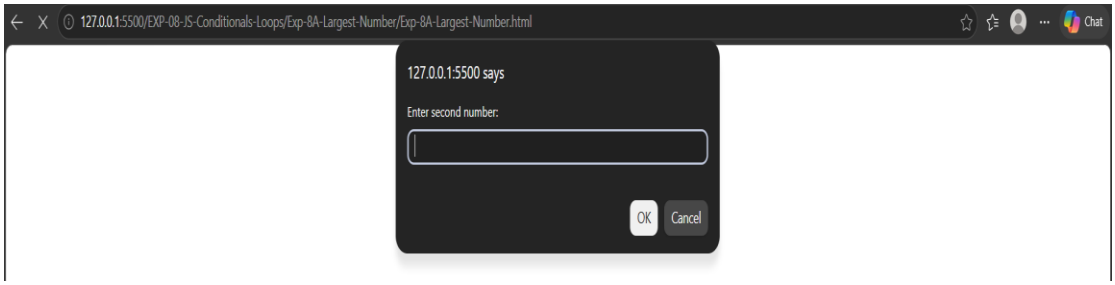
Program:

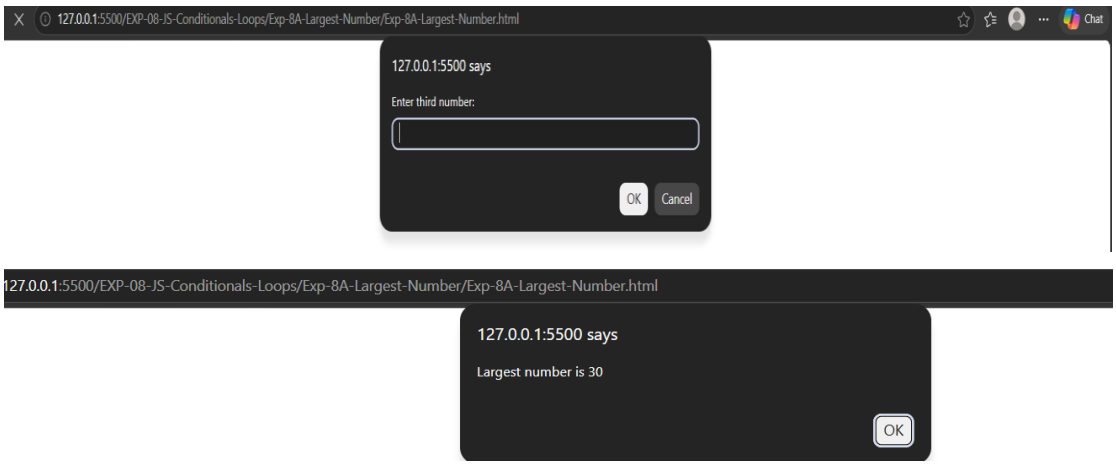
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Largest Number</title>
  </head>
  <body>
    <h1>Largest of Three Numbers</h1>
    <script>
      var a = Number(prompt("Enter first number:"));
      var b = Number(prompt("Enter second number:"));
      var c = Number(prompt("Enter third number:"));
      if (a == b && b == c) {
        alert("Equal numbers");
      } else if (a >= b && a >= c) {
```

```
      alert("Largest number is " + a);
    } else if (b >= a && b >= c) {
      alert("Largest number is " + b);
    } else {
      alert("Largest number is " + c);
    }
  </script>
 </body>
</html>
```

Output:







Experiment: 8b **Date:**

Days of the Week

Write a program to display the days of the week using a switch case.

Aim:

To write a JavaScript program to display the day of the week for a number between 1 and 7 using the switch statement.

Description:

switch statement: Selects one block of code to run from many choices by matching the value of an expression with the case labels.

break: Stops execution after a matching case so that the following cases are not executed.

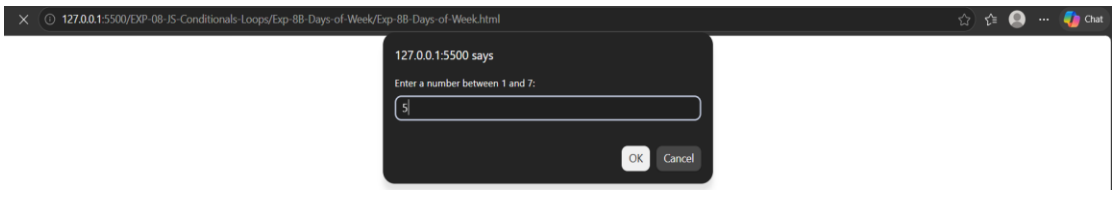
default: Runs when no case matches (here an input outside 1 to 7 gives "Invalid number").

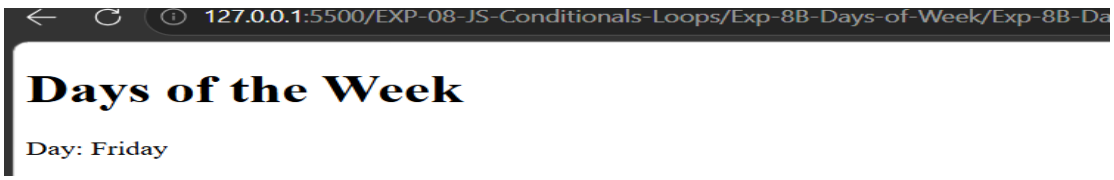
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Days of Week</title>
  </head>
  <body>
    <h1>Days of the Week</h1>
    <script>
      var n = Number(prompt("Enter a number between 1 and 7:"));
      var day;
      switch (n) {
        case 1:
          day = "Monday";
          break;
        case 2:
```

```
        day = "Tuesday";
        break;
      case 3:
        day = "Wednesday";
        break;
      case 4:
        day = "Thursday";
        break;
      case 5:
        day = "Friday";
        break;
      case 6:
        day = "Saturday";
        break;
      case 7:
        day = "Sunday";
        break;
      default:
        day = "Invalid number"   }
    document.write("Day: " + day);
  </script>
 </body>
</html>
```

Output:





Experiment: 8c **Date:**

Loops: 1 to 10

Write a program to print the numbers 1 to 10 using for, while and do-while loops.

Aim:

To write a JavaScript program to print the numbers from 1 to 10 using for, while and do-while loops.

Description:

for loop: Repeats a block of code a fixed number of times; initialization, condition and increment are written in one line.

while loop: Repeats a block of code as long as the condition is true; the condition is checked before each repetition.

do-while loop: Executes the block once and then repeats it while the condition is true; the condition is checked after each repetition.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Loops 1 to 10</title>
  </head>
  <body>
    <h1>Print 1 to 10</h1>
    <script>
      document.write("<h3>Using for loop</h3>");
      for (var i = 1; i <= 10; i++) {
        document.write(i + " ");
      }
      document.write("<h3>Using while loop</h3>");
      var j = 1;
      while (j <= 10) {
```

```
      document.write(j + " ");
      j++;
    }
    document.write("<h3>Using do-while loop</h3>");
    var k = 1;
    do {
      document.write(k + " ");
      k++;
    } while (k <= 10);
  </script>
 </body>
</html>
```

Output:



Experiment: 8d **Date:**

Object Loops

Write a program to print object data using for-in, for-each and for-of loops.

Aim:

To write a JavaScript program to print object data using for-in, forEach and for-of loops.

Description:

for-in loop: Goes through the property names (keys) of an object; the value is accessed as object[key].

forEach(): An array method that calls a function once for every element of the array.

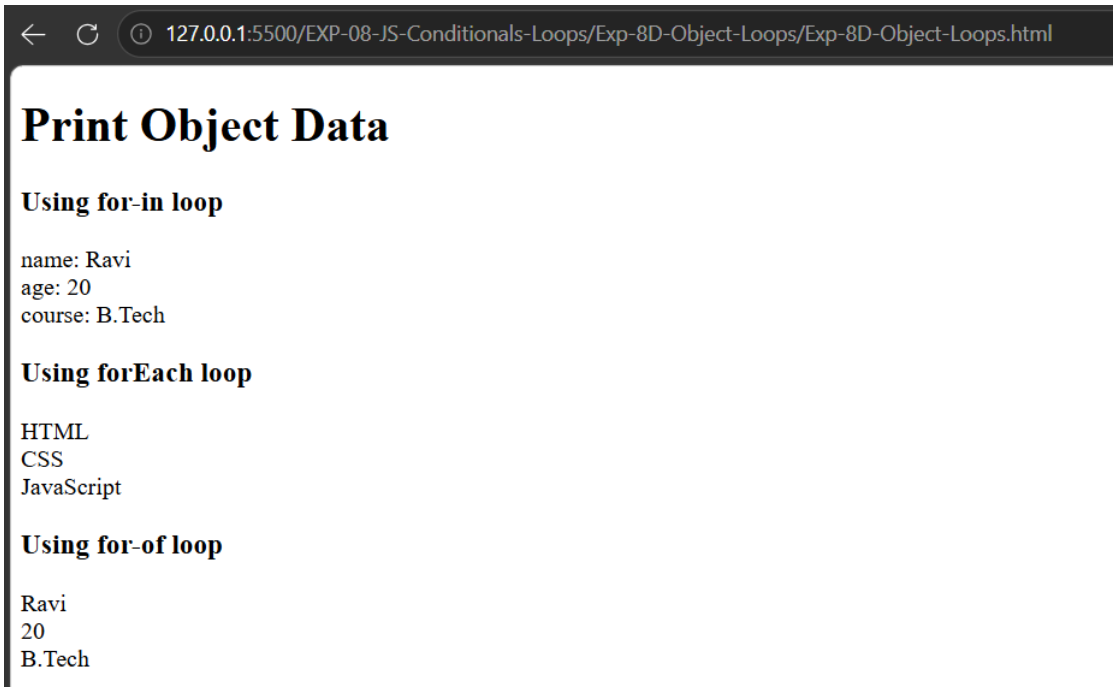
for-of loop: Goes through the values of an iterable such as an array; Object.values() gives the values of an object as an array.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Object Loops</title>
  </head>
  <body>
    <h1>Print Object Data</h1>
    <script>
      var student = {
        name: "Ravi",
        age: 20,
        course: "B.Tech",
      };
      var subjects = ["HTML", "CSS", "JavaScript"];
      document.write("<h3>Using for-in loop</h3>");
      for (var key in student) {
```

```
      document.write(key + ": " + student[key] + "<br>");
    }
    document.write("<h3>Using forEach loop</h3>");
    subjects.forEach(function (item) {
      document.write(item + "<br>");
    });
    document.write("<h3>Using for-of loop</h3>");
    for (var value of Object.values(student)) {
      document.write(value + "<br>");
    }
  </script>
 </body>
</html>
```

Output:



Experiment: 8e **Date:**

Armstrong Number

Write a program to check whether a given number is an Armstrong number (for example 153).

Aim:

To write a JavaScript program to check whether a given number is an Armstrong number.

Description:

Armstrong number: A number that is equal to the sum of the cubes of its digits. For example $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.

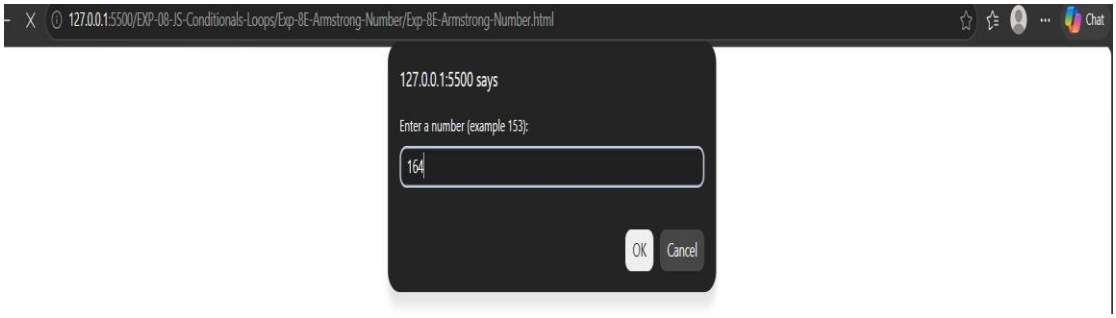
Logic: A while loop extracts each digit using the remainder (%) operator, adds its cube to the sum and removes the digit using Math.floor(temp / 10). Finally the sum is compared with the original number.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Armstrong Number</title>
  </head>
  <body>
    <h1>Armstrong Number Check</h1>
    <script>
      var num = Number(prompt("Enter a number (example 153):"));
      var temp = num;
      var sum = 0;
      while (temp > 0) {
        var digit = temp % 10;
        sum = sum + digit * digit * digit;
        temp = Math.floor(temp / 10);
```

```
    }
    if (sum == num) {
      document.write(num + " is an Armstrong number");
    } else {
      document.write(num + " is not an Armstrong number");
    }
  </script>
 </body>
</html>
```

Output:





Experiment: 8f **Date:**

Denominations

Write a program to show the denomination breakdown (500's, 200's, 100's, 50's, etc.) for an amount deposited.

Aim:

To write a JavaScript program to display the number of notes of each denomination for a deposited amount.

Description:

Logic: The amount is divided by each denomination in descending order (500, 200, 100, 50, 20, 10). Math.floor() gives the number of notes and the remainder (%) is carried to the next denomination.

for-of loop: Goes through the array of denominations one by one.

Remaining amount: The amount left after all the denominations are used (less than 10) is displayed at the end.

Program:

```
<!doctype html>
<html>
  <head>
    <title>Denominations</title>
  </head>
  <body>
    <h2>Deposit Denomination Breakdown</h2>

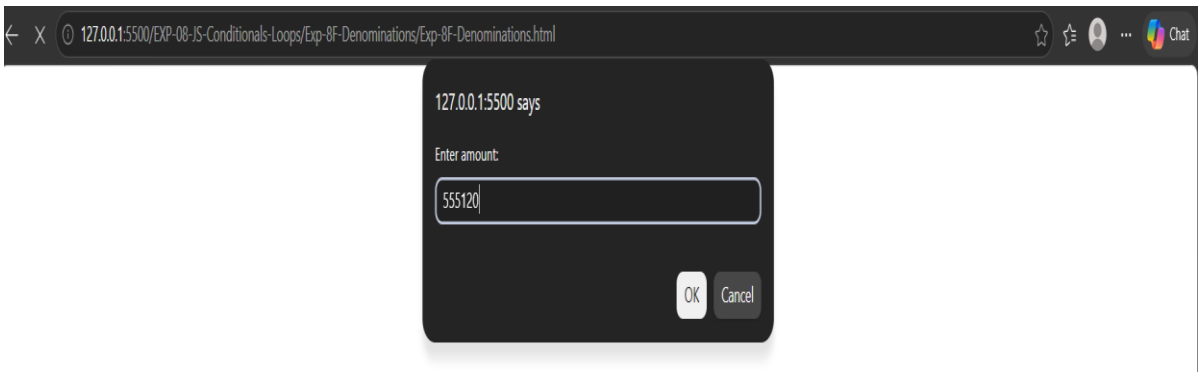
    <script>
      let amount = Number(prompt("Enter amount:"));
      let notes = [500, 200, 100, 50, 20, 10];

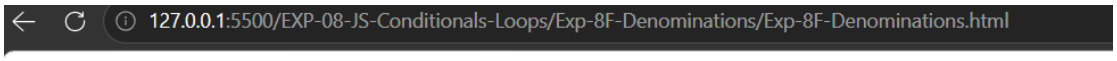
      for (let note of notes) {
        let count = Math.floor(amount / note);
        document.write(note + "'s: " + count + "<br>");
```

```
        amount %= note;
      }

      document.write("Remaining amount: " + amount);
    </script>
  </body>
</html>
```

Output:





Deposit Denomination Breakdown

500's: 1110
200's: 0
100's: 1
50's: 0
20's: 1
10's: 0
Remaining amount: 0

Experiment: 9a **Date:**

JavaScript Functions

Write JavaScript functions for: (i) Factorial, (ii) Fibonacci series, (iii) Prime numbers, (iv) Palindrome check.

Aim:

To write JavaScript functions to find the factorial of a number, generate the Fibonacci series, find prime numbers and check whether a string is a palindrome.

Description:

Function: A block of code that performs a task and is executed when it is called; return sends a value back to the caller.

Factorial: Product of all integers from 1 to n (5! = 120).

Fibonacci series: A series in which each term is the sum of the previous two (0 1 1 2 3 5 …).

Prime numbers: Numbers greater than 1 that are divisible only by 1 and themselves.

Palindrome: A word that reads the same forwards and backwards (e.g. madam); checked by reversing the string with split(), reverse() and join().

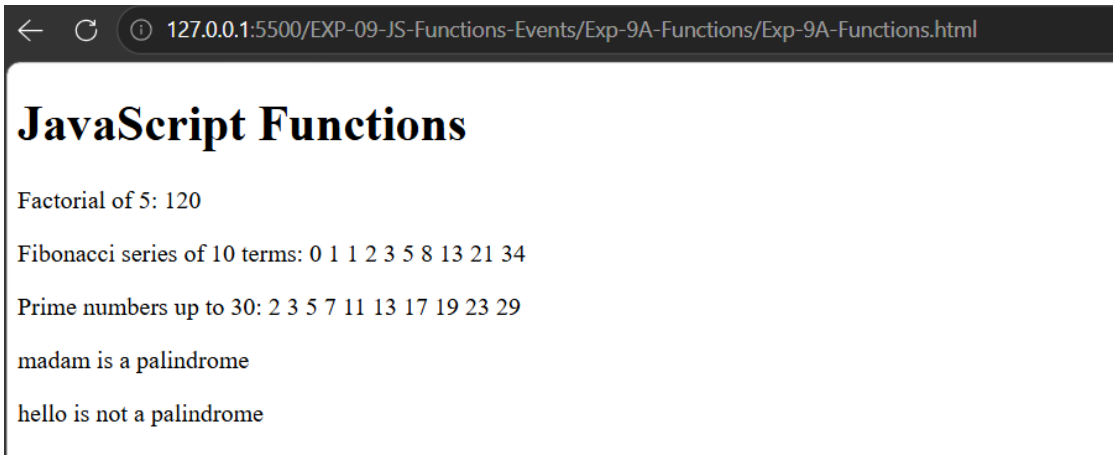
Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>JavaScript Functions</title>
  </head>
  <body>
    <h1>JavaScript Functions</h1>
    <script>
      function factorial(n) {
        var f = 1;
        for (var i = 1; i <= n; i++) {
          f = f * i;
```

```
  }
  return f;
}
function fibonacci(n) {
  var a = 0;
  var b = 1;
  var result = "";
  for (var i = 1; i <= n; i++) {
    result = result + a + " ";
    var c = a + b;
    a = b;
    b = c;
  }
  return result;
}
function prime(n) {
  var result = "";
  for (var i = 2; i <= n; i++) {
    var isPrime = true;
    for (var j = 2; j < i; j++) {
      if (i % j == 0) {
        isPrime = false;
        break;
      }
    }
    if (isPrime) {
      result = result + i + " ";
    }
  }
  return result;
}
function palindrome(s) {
  var rev = s.split("").reverse().join("");
  if (s == rev) {
```

```
      return s + " is a palindrome";
    } else {
      return s + " is not a palindrome";
    }
  }
  document.write("<p>Factorial of 5: " + factorial(5) + "</p>");
  document.write(
    "<p>Fibonacci series of 10 terms: " + fibonacci(10) + "</p>",
  );
  document.write("<p>Prime numbers up to 30: " + prime(30) + "</p>");
  document.write("<p>" + palindrome("madam") + "</p>");
  document.write("<p>" + palindrome("hello") + "</p>");
 </script>
</body>
</html>
```

Output:



Experiment: 9b **Date:**

Functions with Button Events

Create a HTML page with a textbox and four buttons. When a button is clicked, call the function and show the result for: (i) Factorial, (ii) Fibonacci series, (iii) Prime numbers, (iv) Palindrome check.

Aim:

To create a HTML page with a text box and four buttons that call JavaScript functions (using the onclick event) to display the factorial, Fibonacci series, prime numbers and palindrome check for the entered value.

Description:

Event: An action performed by the user, such as clicking a button.

onclick: An event attribute that calls a JavaScript function when the element is clicked, e.g. <button onclick="fact()">.

Reading and showing values: The number typed in the text box is read using document.getElementById("num").value and the result is displayed in a paragraph using innerHTML.

Program:

```
<!doctype html>
<html>
  <head>
    <title>JavaScript Functions and Events</title>
  </head>
  <body>
    <h2>JavaScript Functions</h2>

    <input type="text" id="num" placeholder="Enter a number" />
    <br /><br />

    <button onclick="fact()">Factorial</button>
    <button onclick="fib()">Fibonacci</button>
```

```
<button onclick="prime()">Prime</button>
<button onclick="pal()">Palindrome</button>

<p id="result"></p>

<script>
 function getNum() {
  return parseInt(document.getElementById("num").value);
 }

 function fact() {
  let n = getNum(),
   f = 1;
  for (let i = 1; i <= n; i++) f *= i;
  document.getElementById("result").innerHTML = "Factorial: " + f;
 }

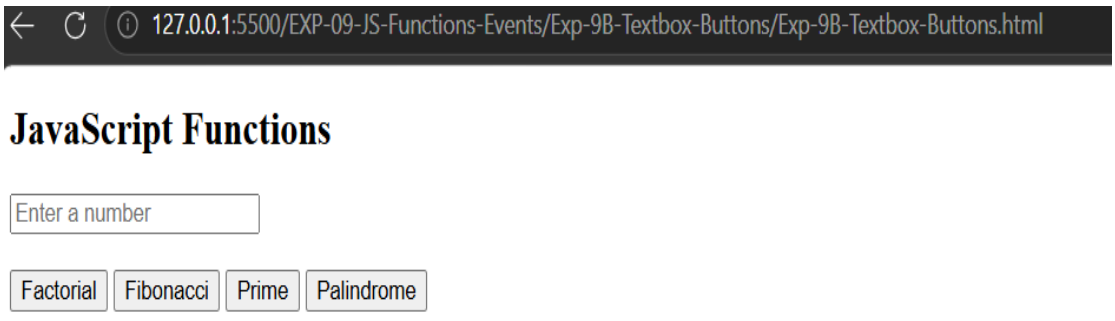
 function fib() {
  let n = getNum(),
   a = 0,
   b = 1,
   s = "";
  for (let i = 1; i <= n; i++) {
   s += a + " ";
   let c = a + b;
   a = b;
   b = c;
  }
  document.getElementById("result").innerHTML = "Fibonacci: " + s;
 }

 function prime() {
  let n = getNum(),
   s = "";
```

```
      for (let j = 2; j <= n; j++) {
        let p = true;
        for (let i = 2; i < j; i++) if (j % i == 0) p = false;
        if (p) s += j + " ";
      }
      document.getElementById("result").innerHTML = "Primes: " + s;
    }

    function pal() {
      let n = document.getElementById("num").value;
      let r = n.split("").reverse().join("");
      document.getElementById("result").innerHTML =
        n === r ? "Palindrome" : "Not Palindrome";
    }
  </script>
 </body>
</html>
```

Output:





Experiment: 9c **Date:**

Registration Form Validation

Validate a registration page: (i) Name (must start with a letter, alphanumeric, minimum 6 characters), (ii) Mobile (10 digits), (iii) Email (xxxx@xxxx.xxx).

Aim:

To validate a registration page using JavaScript regular expressions for the name, mobile number and e-mail fields.

Description:

Form validation: Checking the data entered by the user before the form is submitted. The onsubmit event calls validate(); returning false stops the submission.

Name: /^[A-Za-z][A-Za-z0-9]{5,}$/ – starts with a letter, followed by at least 5 letters or digits (minimum 6 characters).

Mobile: /^[0-9]{10}$/ – exactly 10 digits.

Email: /^\w+@\w+\.\w+$/ – of the form xxxx@xxxx.xxx.

test(): The regular-expression method that checks whether a value matches the pattern.

Program:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Registration Validation</title>
  </head>
  <body>
    <h1>Registration Page</h1>
    <form onsubmit="return validate();">
      <table cellpadding="8">
        <tr>
          <td>Name</td>
          <td><input type="text" id="name" /></td>
        </tr>
```

```
        <tr>
          <td>Mobile</td>
          <td><input type="text" id="mobile" /></td>
        </tr>
        <tr>
          <td>Email</td>
          <td><input type="text" id="email" /></td>
        </tr>
        <tr>
          <td colspan="2"><input type="submit" value="Register" /></td>
        </tr>
      </table>
    </form>
    <script>
      function validate() {
        var name = document.getElementById("name").value;
        var mobile = document.getElementById("mobile").value;
        var email = document.getElementById("email").value;
        var namePattern = /^[A-Za-z][A-Za-z0-9]{5,}$/;
        var mobilePattern = /^[0-9]{10}$/;
        var emailPattern = /^\w+@\w+\.\w+$/;
        if (!namePattern.test(name)) {
          alert(
            "Name must start with a letter, be alphanumeric and have at least 6
characters",
          );
          return false;
        }
        if (!mobilePattern.test(mobile)) {
          alert("Mobile number must have exactly 10 digits");
          return false;
        }
        if (!emailPattern.test(email)) {
          alert("Email must be in the format xxxx@xxxx.xxx");
```

```
        return false;
      }
      alert("Registration successful");
      return true;
    }
  </script>
 </body>
</html>
```

Output:

